

legacy-reverse 設計・仕様書

legacy-reverse project

目次

1	全体像	4
1.1	何を解決するシステムか	4
1.2	設計原則(5つ)	4
1.3	システムの3+1層	5
1.4	整合性の背骨: ハッシュ連鎖	6
1.5	品質保証の分担(全体)	6
1.6	運用の全体像	6
1.7	読み方ガイド	7
2	アーキテクチャと設計判断	9
2.1	責務の分離: skills / scripts / MCP / hooks / 画面	9
2.2	フェーズ・スキルの構成	10
2.3	クリーンルーム分業の意図	10
2.4	主要コンポーネントの責務(scripts)	12
2.5	スケール設計(2000 関数級)	13
2.6	拡張ポイント	13
3	パイプライン仕様(①～⑦)	15
3.1	フェーズ一覧(全体)	15
3.2	情報遮断表	16
3.3	各フェーズの詳細仕様	16
4	データ仕様	19
4.1	データ設計の原則(全体)	19
4.2	対象プロジェクトのディレクトリ構成	19
4.3	data/functions.json	20
4.4	data/ledger.json	22
4.5	.legacy-reverse/state.json	22
4.6	.legacy-reverse/pipeline-status.json	22
4.7	.legacy-reverse/batch-priority.json	23
4.8	フロントマターの status 遷移(完了判定の正)	23
4.9	WBS の表示記号	24
5	品質ゲート仕様	25
5.1	全体像	25
5.2	機械で取れないもの、の分担	25

5.3	review_checks.py の検査仕様	26
5.4	承認経路の強制(サーバ側の再検証)	27
5.5	hooks: 拒否と差し戻しの 2 系統	27
5.6	スタブ検知: check_stubs.py	27
5.7	⑤の制御信号(collect_results.py の exit code)	28
5.8	完了検証: ledger check(⑥)	28
6	画面設計	29
6.1	画面が担うもの	29
6.2	配信アーキテクチャ	30
6.3	HTTP エンドポイント仕様	31
6.4	ウィジェット — 状態に応じて画面に現れる操作 UI	33
6.5	画面別仕様	34
6.6	状態ファイルと排他	37
6.7	中止・スキップの 2 系統	38
6.8	レンダリング経路	39
6.9	設計判断のまとめ	39
7	MCP サーバ仕様	41
7.1	設計方針(全体)	41
7.2	ツールリファレンス(30)	41
7.3	実装上の契約	45
8	運用設計	46
8.1	運用モデル(全体)	46
8.2	導入手順	46
8.3	日常運用	47
8.4	無人バッチ実行	49
8.5	中断・再開	52
8.6	大規模運用(200 関数超)	52
8.7	配布・出力	52
8.8	手編集の可否	53
8.9	トラブルシューティング(運用者向け要約)	53

1. 全体像

i この文書の位置づけ

本サイトは legacy-reverse の**設計・仕様**を扱う。「どう使うか」は既存の利用者向けドキュメント (スライド版チュートリアル・QUICKREF・MANUAL の HTML/PDF) が担い、本サイトは「なぜこの構造か・各部品は何を保証するか・どう運用するか」を、このページ(全体)→各章(詳細)の順で読めるようにしたもの。

1.1 何を解決するシステムか

レガシーコード(Fortran / C・C++ 等)を Python へ**仕様ベースで移植**するための Claude Code スキル群+決定的スクリプト群+MCP サーバ。「レガシーを読んで直接書き写す」のではなく、**関数仕様書という中間成果物**を挟み、独立に作ったテストと実装を突き合わせることで品質を機械的に担保する。

パイプラインは 8 フェーズ。**最初のトリガは必ず人が引く**(AI が勝手にフェーズを進めることはない)。連続実行を開始した場合だけ、その 1 回のトリガのあと工程間の 遷移が自動になる — ただし承認・裁定は必ず人を待つ。

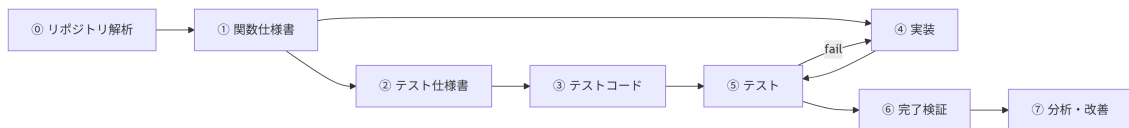


図 1.1: 8 フェーズの依存関係と④⑤の反復ループ

要点は**①仕様書だけを共通の親とする 2 系統**(テスト系 ②③、実装系 ④)を ⑤で突き合わせること。②③④は互いを見ず、レガシー原文も見ない。仕様書の曖昧さ・穴は、必ずテスト失敗または spec-gap ISSUE として表面化する。

1.2 設計原則(5 つ)

表 1.1: 設計原則と、それぞれが防ぐもの

原則	内容	それで防ぐもの
クリーンルーム分業	②③はレガシー原文を見ない。 ③は②だけ、④は①だけを入力にする	「レガシーの写経」による仕様の暗黙化
状態はすべてファイル	進捗の正は functions.json・ledger.json・成果物フロントマター。会話コンテキストに置かない	セッション切れによる「1 からやり直し」
機械が正、AI は意味づけ	列挙・検証・台帳操作は決定的スクリプト。AI は読解と執筆だけ	ハルシネーション・省略・数え漏れ

原則	内容	それで防ぐもの
人の承認ゲート	仕様確定・テスト承認・裁定は人。AI は「仮説+Yes/No」で聞く	AI による勝手な仕様確定
強制は仕組みで	テスト保護は hook、スタブ検知・引用検証はスクリプト	「プロンプトで禁止」の不確実さ

詳細: アーキテクチャと設計判断

1.3 システムの 3+1 層

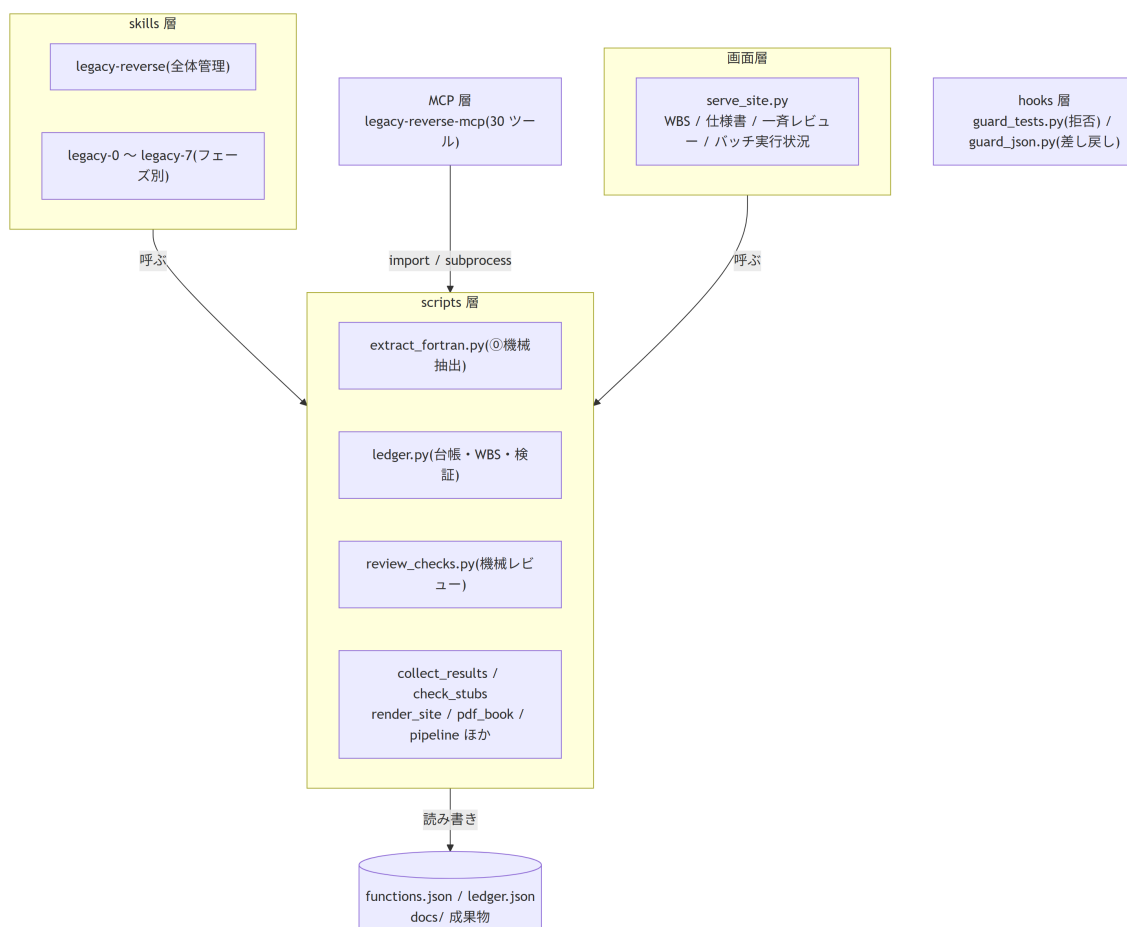


図 1.2: skills / scripts / MCP / hooks / 画面の呼び出し関係

表 1.2: 各層の役割と、詳細を書いた章

層	役割	章
skills	「いつ・何を・どの順で」を書いた手順書。レガシー読解・仕様執筆・トリアージの 判断だけ を LLM が行う	アーキテクチャ / パイプライン仕様
scripts	単体でも動く決定的処理。列挙・検証・台帳・レンダリングの 正	品質ゲート仕様 / データ仕様

層	役割	章
MCP	scripts の型付きラッパ(判断は持たない・二重管理なし)。許可プロンプトとシェル事故を減らす	MCP サーバ仕様
hooks	LLM の自制に頼らない強制層。PreToolUse は 拒否 、PostToolUse は 差し戻し	品質ゲート仕様
画面	人の 4 つの仕事(トリガ・回答・承認・裁定)をブラウザで完結させる操作面	画面設計

1.4 整合性の背骨: ハッシュ連鎖

①→②→③ の各成果物は上流のハッシュを記録する。上流が改訂されると下流の照合が落ち、WBS に stale△ / △改変が自動表示される。「①を直したのに②を直し忘れた」は構造的に起きない。

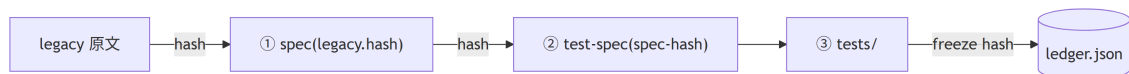


図 1.3: ①→②→③ と台帳に伝搬するハッシュ連鎖

詳細: データ仕様 — ハッシュ連鎖と status 遷移

1.5 品質保証の分担(全体)

LLM 成果物は人に届く前に、決定的スクリプトの検証を必ず通る。

表 1.3: 誤りの種類と、それを検知する仕組み

誤りの種類	検知する仕組み
数え漏れ(⑥)	抽出器内蔵の 2 系統カウント突合
実在しない根拠の引用・省略(①)	review_checks.py spec
仕様 ID の捏造・ケース漏れ(②)	review_checks.py testspec
ケースとテスト関数の過不足(③)	マーカー突合(collect_results exit 3)
スタブ実装(④)	check_stubs.py
上流改訂の見落とし(⑤前)	ledger verify
意味レベルのすり替え	機械では取れない → 人の①レビュー+⑤の突き合わせが受け持つ

詳細: 品質ゲート仕様

1.6 運用の全体像

人の仕事は 4 つだけ — トリガを引く・質問に答える・承認する・裁定する。この 4 つはすべてチャットからでもブラウザからでも行える。

運転モードは 5 経路あり、規模と「どこまで自動で進むか」で選ぶ。

表 1.4: 5 つの運転モードと、それぞれが向く規模

経路	起動	進む範囲	向く規模
対話	チャット <code>/legacy-N-xxx</code>	1 関数 × 1 工程	数件・様子を見ながら
会話バッチ	チャット「まとめて N 件」	①を N 件 → 一斉レビュー表	数十件
ブラウザ単発	仕様書ページのボタン	1 関数 × 1 工程	数件
ブラウザ連続実行	バッチ実行状況ページ	①～⑤を工程横断で自動	数十～数百件
無人バッチ (CLI)	<code>pipeline.py spec</code>	①のみ・全件	数百～2000 件

- ・ブラウザ連続実行と無人バッチはどちらも 1 関数 = 1 headless プロセスなので、会話のトークン上限に依存しない。両者は同じ実行枠(ロック)を共有し二重起動しない
- ・進捗の閲覧と操作は WBS の HTML サイト(`serve_site.py`)。レビュアーへは **単体実行 EXE** を配布する(こちらは閲覧専用)
- ・中断・再開はいつでも安全。再開手順は常に同じ 3 コマンド(summary → next → review all)
- ・2000 関数級では WBS が自動でダッシュボード+ファイル別明細に分割される

詳細: 運用設計 / 画面設計

1.7 読み方ガイド

表 1.5: 知りたいことから読む章を引く対応表

知りたいこと	読む章
なぜこの構造か・設計判断の理由	アーキテクチャと設計判断
各フェーズの入出力・完了条件・情報遮断	パイプライン仕様
<code>functions.json</code> / <code>ledger.json</code> / フロントマターの形	データ仕様
機械レビュー・hook・スタブ検知が何を保証するか	品質ゲート仕様
どの画面に何が出るか・HTTP API・承認 UI の設計	画面設計
MCP ツール 30 個の入出力仕様	MCP サーバ仕様
導入・日常運用・無人バッチ・配布・トラブル対処	運用設計

1.7.1 既存ドキュメントとの関係

表 1.6: 既存ドキュメントと本サイトの守備範囲

文書	対象読者	本サイトとの関係
<code>slides/index.html</code>	初見の操作者	操作のチュートリアル(⑥→⑦体験)
<code>QUICKREF.md</code>	作業中の操作者	コマンド即引き 1 枚

文書	対象読者	本サイトとの関係
MANUAL.md / MANUAL.html / MANUAL.pdf	操作者	背景と操作の意味・トラブル対処。HTML は画像込みの単一ファイルで配れる
本サイト	設計者・保守者・運用管理者	構造・仕様・運用設計の正を集約 (DESIGN.md を包含・拡張)
references/ workflow.md / schema.md	フェーズ skill・スクリプト	実行時規則とデータの正(機械が従う原本)

2. アーキテクチャと設計判断

2.1 責務の分離: skills / scripts / MCP / hooks / 画面

本システムの最重要の設計判断は、「判断」と「機械処理」を層で分けたこと。

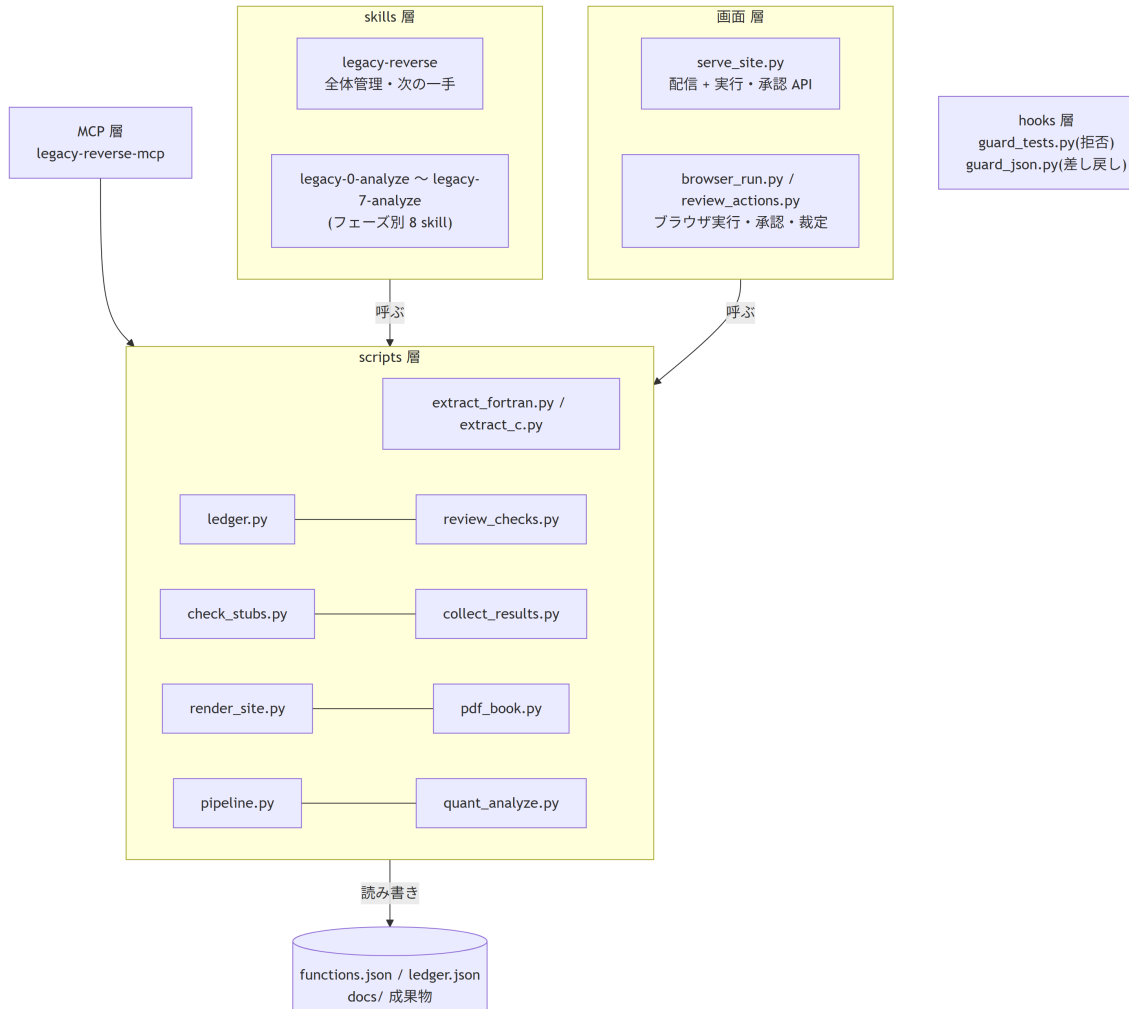


図 2.1: skills / scripts / MCP / hooks / 画面の責務分担

表 2.1: 各層の中身と、そう分けた理由

層	中身	設計判断
skills	「いつ・何を・どの順で」の手順書。 実作業の判断(レガシー読解・仕様執筆・トリアージ)だけを LLM が行う	LLM は数え上げ・照合・台帳操作を すると誤る(ハルシネーション・省略)。 判断以外を持たせない
scripts	単体でも動く決定的処理	列挙・検証・状態管理は再現性が命。 LLM を通さない

層	中身	設計判断
MCP	scripts を import/subprocess するだけの型付きラッパ	二重管理なし。ロジックは常に scripts 側が正
hooks	PreToolUse は呼び出しを拒否、PostToolUse は結果を検証して差し戻す	「プロンプトで禁止」は破られ得る。強制は仕組みで
画面	人の操作(トリガ・承認・裁定)の入口。判断もロジックも持たない	LLM に渡すツールではないので MCP 化しない。実行の実体は scripts 側と共有する

skills から scripts への呼び出しは、MCP 登録済み環境では **MCP ツール優先**、未登録なら直接実行。どちらでも実体は同一(運用設計 ― 導入参照)。

実行の入口は 3 つあるが実体は 1 本。チャットの skill・CLI の無人バッチ・ブラウザのボタンは、いずれも `pipeline.run_one`(headless Claude 1 回 + ファイル状態での契約検証)に合流し、同じ実行枠ロックを取り合う。入口を増やしても実行系統は分岐しない(画面設計)。

2.2 フェーズ・スキルの構成

スキルは「全体管理 1 + フェーズ別 8」に分割されている。フェーズごとに分けるのは、(1) 情報遮断を「そのフェーズの skill には読んではいけない入力 が 書かれていない」形で構造化するため、(2) 人がトリガを引く単位と一致させるため。

表 2.2: フェーズ別スキルの担当範囲と入力の制限

skill	フェーズ	責務
legacy-reverse	—	セットアップ確認・進捗要約・次アクション提案(状態はスクリプトに聞く)
legacy-0-analyze	①	解析 → functions.json・骨子・WBS・規約(Fortran の列挙は機械抽出。LLM は説明の充填とレポート確認のみ)
legacy-1-spec	①	仕様書執筆。 レガシーを読める唯一の役割 (spec-gap 改訂も担当)。バッチモードあり
legacy-2-testspec	②	テスト仕様。①のみ入力。人の承認ゲート
legacy-3-testcode	③	テストコード。②のみ入力。完了時 freeze でハッシュ固定
legacy-4-impl	④	実装。①のみ入力。スタブ禁止
legacy-5-test	⑤	実行・結果収集・トリアージ・④⑤ループ管理
legacy-6-check	⑥	完了検証と最終レンダリング
legacy-7-analyze	⑦	分析・改善(挙動保存。テスト全 pass 維持が絶対条件)

2.3 クリーンルーム分業の意図

②と④が互いを見ない・レガシーを見ないことが品質保証の核。

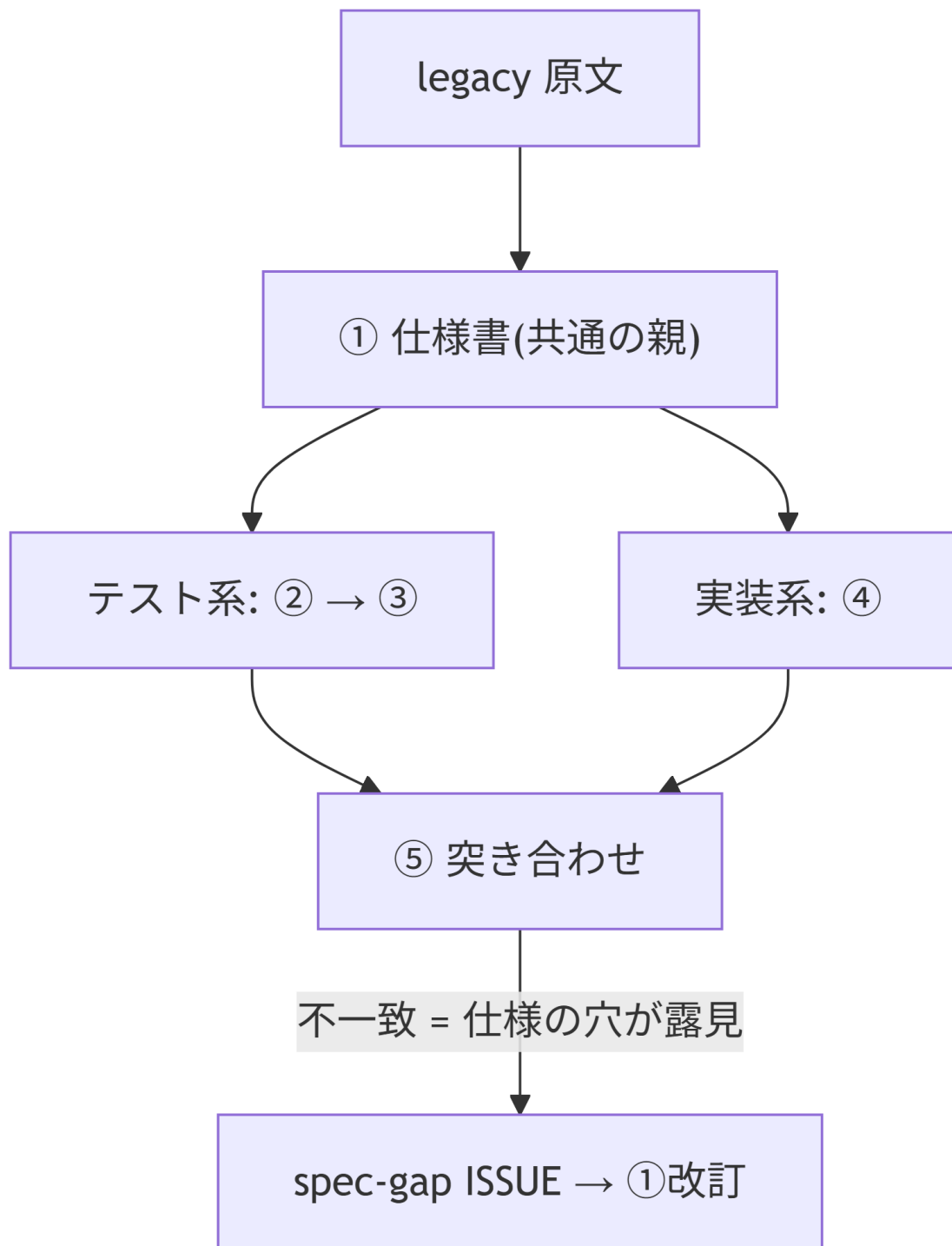


図 2.2: ①を共通の親とする2系統と⑤での突き合わせ

- ③が①やレガシーを見られると「実装に合わせたテスト」が書ける(=検証にならない)
- ④がレガシーを見られると「写経」になり、仕様書に書かれていない暗黙挙動が残る
- どちらも遮断すると、**仕様書の穴は⑤で機械的に(テスト失敗として)露見する**
- ④が詰まったとき自力でレガシーを見に行くのではなく、spec-gap ISSUE を起票 → ①改訂 エージェント(レガシーを読める唯一の役割)が仕様を更新 → ハッシュ伝搬 → ④再開

遮断表の正は references/workflow.md。パイプライン仕様に転載。

2.4 主要コンポーネントの責務(scripts)

表 2.3: 主要スクリプトの責務と設計メモ

ファイル	責務	設計メモ
extract_fortran.py	Fortran 静的解析 → functions.json 生成/マージ	固定・自由形式、継続行、COMMON/USE/intent、call+関数参照の呼出推定。 再実行 = マージ(func_id 不変・手修正保持) が再開性の要
extract_c.py	C/C++ 静的解析 → 同じ functions.json にマージ	Fortran↔C の呼び出しをアンダースコア規約込みで突合。 抽出の実行順に依存しない (後から走った側が未解決名を解決する)
ledger.py	台帳・状態判定・WBS・骨子・検証・⑥check	状態判定 <code>status_of()</code> が唯一の判定ロジック(WBS / next / check が共有)。200 関数超で WBS を自動分割。 正データが壊れていたら行番号つきで停止する (黙って空扱いにしない)
review_checks.py	①②の機械レビュー・一斉レビュー表の生成	LLM 不使用。書式はテンプレ(assets/templates)に依存。表は「承認できる行」が上に来る順で出す
render_site.py	docs/ → HTML サイト・ウィジェットの焼き込み	Quarto が Mermaid を .qmd でしか描けないため影コピー(_sitework)方式。 差分レンダが既定 (2000 関数で数十秒)。承認・実行ボタンはここで各ページに埋め込まれる = UI は再レンダ時点のスナップショット
pipeline.py	無人バッチドライバ・実行ループの実体	1 関数 = 1 headless Claude プロセス (毎回まっさらなコンテキスト = トークン上限に依存しない)。完了は LLM の申告でなくファイル状態で契約検証。中断安全・コスト集計・連続失敗停止。 <code>run_one</code> は 3 つの入口が共有する
browser_run.py	ブラウザからの単発実行・連続実行・残タスク走査	<code>_decide_kind</code> が「次に着手すべき 1 工程」を決める唯一の判定。連続実行は 1 件ごとに再走査するので、承認が下りた関数を自動で拾う
review_actions.py	①②の承認・修正依頼、⑤の裁定	承認時にサーバ側で機械レビューを再実行してから確定 (クライアント表示を信用しない)。完了後に WBS・一斉レビュー表・サイトを再生成

ファイル	責務	設計メモ
serve_site.py	ローカル配信 + 実行・承認 API	127.0.0.1 のみ・プロジェクト別 固定ポート・ <code>--watch</code> 。書き込み POST は 4 段のガード (接続元・ Host/Origin・EXE 判定・ルート 限定)。人の操作の入口なので MCP 化しない
collect_results.py	⑤結果収集 → 報告書生成	exit code が制御信号(0 pass / 1 fail / 2 blocked / 3 mismatch)。 attempt 上限 3 で自動 block
check_stubs.py	④のスタブ検出	空実装・ NotImplementedError・TODO/ FIXME
quant_analyze.py	⑦の定量計測(LLM 不使用)	cProfile + radon / ruff / bandit / pip-audit を機械実行し quant.json・quant.md に残す。 計測せずに提案しないための 1 段目
pdf_book.py	種別ごとの合本 PDF	quarto-typst-pdf skill に委譲
build_viewer.py	配布用単体実行ファイル	サイト+配信サーバを PyInstaller で 1 ファイル化。受 け手に Python / Quarto 不要。 閲覧専用 (書き込み API は 403)
hooks/ guard_tests.py	④⑤中の tests/ 編集拒否 (PreToolUse)	state.json(phase-start / end)を 見て判定
hooks/ guard_json.py	編集直後の JSON 構文検証 (PostToolUse)	壊れていたら exit 2 で エージェ ント自身に差し戻す 。正データ の破損でバッチが止まるのを源 流で断つ

2.5 スケール設計(2000 関数級)

会話コンテキストに全関数リストを載せない、が一貫した方針。

- **WBS の自動 2 段構成**: 200 関数以下は 1 ページ。超えるとトップが「要対応・次の一手・ファイル別進捗」のダッシュボードになり、明細は docs/wbs/ に分割
- **軽量 API**: `ledger status --summary`(数行の JSON 要約)と `ledger next --all`(着手可能リスト)。再開・状況把握はこの 2 つで足りる
- **再開手順は常に同じ**: `summary` → `next` → `review all`。⑨の再抽出もマージなので安全
- 呼び出し推定は全 function 名を 1 本の交替正規表現に畳んで走査($O(n^2)$ 回避)
- ①の全件実行は会話内ループ禁止(コンテキスト上限・コンパクション劣化)。pipeline.py で 1 関数 1 プロセスに分解する

2.6 拡張ポイント

表 2.4: 拡張ポイントと、追加するときの手順

拡張	方法
新レガシー言語(C# 等)	extract_fortran.py / extract_c.py と同じ出力契約 (functions.json スキーマ + extract-report)で抽出器を追加し、MCP ツールに登録。既存の抽出結果とはマージされ、実行順は問わない
新フェーズの自動運転化	①～⑤は実装済み(<code>browser_run.KINDS</code> に「プロンプト・検証関数・リトライ数」を1行足す形)。⑥は LLM 不使用の同期実行、⑦は計測 → 提案の2段。 新しい工程を足すときも骨格は同じ : 対象選定 → headless 実行 → ファイル状態で契約検証 → ログ
意味レベルのレビュー強化	別コンテキストのレビューエージェント(①と legacy だけを読む)をドライバ段に挿入
⑦の改善適用ループ	現状は「計測 → 提案」まで。施策票の承認 → 適用 → 再計測 → 達成判定のイタレーションは形が大きく異なるため別設計
新言語ターゲット	現状 Python+pytest。将来 JS / Rust 拡張を想定

関連: 画面設計 — 実行・承認の入口としての画面層の詳細

3. パイプライン仕様(①～⑦)

3.1 フェーズ一覧(全体)

最初のトリガは必ず人が引く。AI が勝手に次フェーズへ進むことはない。起動の入口は3つあるが、実行の実体(headless Claude 1 回 + ファイル状態での契約検証)は 共通で、同じ実行枠ロックスを取り合う。

表 3.1: ①～⑦の起動方法と入出力および完了判定

フェーズ	起動(チャット)	起動(ブラウザ)	入力	出力	完了判定(機械)
① 解析	/legacy-0-analyze	—	legacy/ 全部	functions.json・ 骨子・WBS・規約	抽出器の完全性突 合
② 仕様書	/legacy-1-spec F-xxxx	仕様書 ページ / 連続実行	legacy/ 該当関数 +DK	docs/specs/*.md	status: reviewed(人 OK)
③ テスト仕様	/legacy-2-testspec F-xxxx	仕様書 ページ / 連続実行	①(reviewed)規約	docs/test-specs/*.md	status: approved かつ spec-hash 一 致
④ テストコード	/legacy-3-testcode F-xxxx	仕様書 ページ / 連続実行	②(approved)規約	tests/	ledger の freeze ハッシュ一致
⑤ 実装	/legacy-4-impl F-xxxx	仕様書 ページ / 連続実行	①(reviewed)規約	src/	モジュール存在か つスタブ検知ゼロ
⑥ テスト	/legacy-5-test F-xxxx	仕様書 ページ / 連続実行	実行結果 +①②	docs/test-results/	result: pass(実装率 100%・失敗 0)
⑦ 完了検証	/legacy-6-check	WBS トップ(全⑤完了後)	全成果物	completion-check.md	ledger check + review all が exit 0
⑧ 分析・改善	/legacy-7-analyze	WBS トップ(⑦pass後)	src/・ docs/・ 計測	analysis.md・施 策票	テスト全 pass 維持 (挙動保存)

連続実行は①～⑤を工程横断で自動的に回す(1 件ごとに全関数を再走査し、次に着手できる工程を選ぶ)。承認待ち・裁定待ちはスキップするので、自動で進む範囲と人が要る範囲がそのまま分かれる。詳細は 画面設計。

3.2 情報遮断表

各フェーズが読んでよい入力(正)の正(references/workflow.md より)。「読んではいけない」に触れたくなったら、それは仕様の穴 — ISSUE を起票して停止する。

表 3.2: フェーズごとに読んでよい入力と禁止する入力

フェーズ	読んでよい入力	読んではいけないもの
⑥ 解析	legacy/ 全部	—
① 仕様書	legacy/ 該当関数、functions.json、domain-knowledge.md	tests/、src/
② テスト仕様	①(reviewed)、conventions.md、domain-knowledge.md	legacy/、src/、tests/
③ テストコード	②(approved)、conventions.md	legacy/、①、src/
④ 実装	①(reviewed)、conventions.md	legacy/、②、tests/
⑤ テスト	結果+①②(トリアージ判断用)、src/((a)修正時)	legacy/、tests/ の編集
⑦ 分析	src/・docs/・計測結果(全体を見る)	tests/ の編集(挙動保存が大原則)

レガシー原文を読める役割は ⑥ と ①(改訂含む)だけ。

3.3 各フェーズの詳細仕様

3.3.1 ⑥ リポジトリ解析

- 人が答えること: レガシーの場所・言語・新パッケージ名。型対応表・規約を一緒に確定
- **Fortran の関数列挙は静的解析プログラム(extract_fortran.py)が行う**。LLM の仕事は説明の充填と抽出レポート(data/extract-report.json)の確認のみ
- 抽出器は固定・自由形式、継続行、COMMON/USE/intent、call+関数参照による呼出推定に対応
- **再実行は常にマージ**: func_id 不変・手修正保持・新規のみ追加。途中でやり直しても 採番のやり直しや成果物の宙吊りは起きない
- 完全性突合(2 系統カウントの照合)が NG のファイルは監査ログに残り、調査対象になる
- 出力: functions.json(正データ)・仕様書骨子・WBS・conventions.md・docs-sphinx/ テンプレ配置

3.3.2 ① 関数仕様書

- 執筆単位は関数。仕様の各項目に **Confidence**(● 確認済 / ● 推測 / ● 仮定)と **レガシー行番号の根拠**を必須付与
- status 遷移: skeleton → draft → **reviewed(人が OK して確定)**
- draft 化の直後に機械レビュー(review_checks.py spec)を通し、NG ゼロにしてから人に出す
- ● ● 項目は ISSUE(仮説+Yes/No)として人へ。回答は domain-knowledge.md に蓄積され再質問を防ぐ
- **バッチモード**: 「まとめて N 件」で複数関数を draft まで連続処理し、一斉レビュー表(docs/spec-review.md)で人がまとめて処理する。表は「承認できる → ISSUE あり → AI 修正待ち」の順に並び、行内のボタンで承認・修正依頼が完結する(画面設計)
- **全件(数百～2000 件)は無人ドライバ pipeline.py で回す**(運用設計参照)。ブラウザの連続実行でも同じことができ、そちらは①以降の工程も続けて進む
- draft の書き直しは自由。reviewed の書き直しは②が stale になるため人の了承を先取る

- **spec-gap 改訂**: ④が詰まって起票された ISSUE への対応も①の担当 (レガシー確認・根拠付きで仕様を更新 → ハッシュ伝搬で下流に再生成が波及)

3.3.3 ② テスト仕様書

- 入力は ①(reviewed)+規約+DK のみ。**レガシーは読まない**
- ●の仕様項目すべてにテストケースを対応付ける (トレーサビリティ)。ケース ID(TC-xxx)と SPEC-ID の対応が機械レビューで突合される
- レガシー実行環境なしが基本前提のため、期待値は「仕様 ●+人間確認」で確定 (実測は環境がある場合のみ)
- フロントマター `spec-hash` に生成時点の①ハッシュを記録 (連鎖の起点)
- status 遷移: generated → **approved(人の承認ゲート)**。⚠未確定ゼロが承認条件
- 承認依頼前に `review_checks.py testspec` を通し NG ゼロにする

3.3.4 ③ テストコード

- 入力は ②(approved)+規約のみ。①もレガシーも読まない
- ②の全ケースを `pytest` 関数に実装し、TC マーカーで対応付ける (`pytest --collect-only`+マーカー突合で過不足を機械検知)
- 完了時に `ledger freeze-tests` でテストファイルのハッシュを台帳に記録。以降の改変は「⚠改変」として機械検知される
- ④⑤中の tests/ 編集は hook が物理的に拒否。テスト側の疑義は ISSUE → 人承認 → ②③再生成が正規経路

3.3.5 ④ 実装

- 入力は ①(reviewed)+規約のみ。②③もレガシーも読まない
- **スタブ量産の禁止**: 空実装・NotImplementedError・TODO/FIXME は `check_stubs.py` が検出し未完了扱い
- 実装しきれない場合は spec-gap ISSUE を起票して停止 → ①改訂 → ハッシュ伝搬 → ④再開。自力でレガシーを見に行くことは構造上できない (遮断+hook)
- 詳細仕様は docstring に書き、Sphinx で API ページ化 (トレース対応表で仕様書と相互リンク)

3.3.6 ⑤ テスト

実行フロー (run_tests は⑤の一括 MCP ツール):

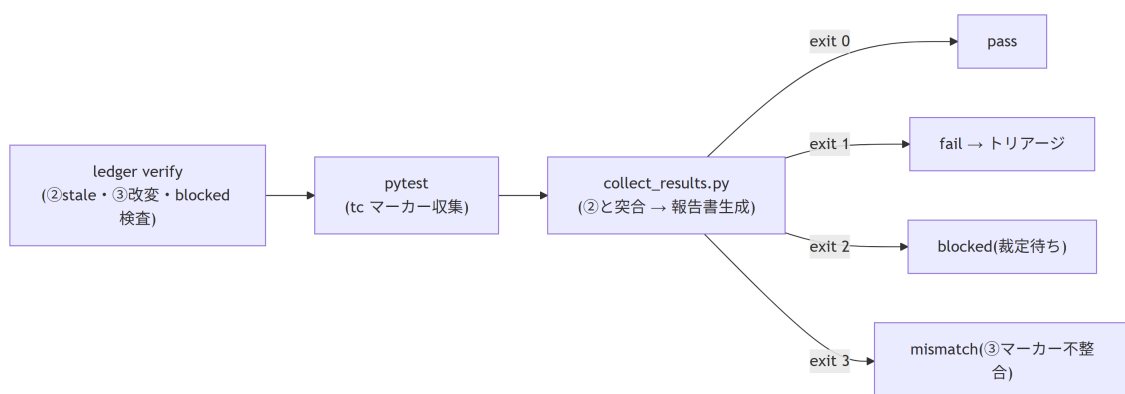


図 3.1: ⑤テストの実行フローと exit code による分岐

- pass 条件は「**実装率 100%**(②の全ケースが③に実装済み)かつ**失敗 0**」
- 結果報告書は `{func-id}_{YYYYMMDD-HHMMSS}.md` で毎回 1 から自動生成 (履歴はファイルとして蓄積)
- fail 時のトリアージは 3 分類。(a) だけ AI が自走し、(b)(c) は人の承認が要る:

表 3.3: ⑤失敗時のトリアージ 3 分類と対応の分担

分類	意味	対応
(a) 実装バグ	実装が①を満たしていない	AI が④修正 → ⑤再実行を自走(上限 3 回)
(b) テストコードバグ	③が②とズレている	ISSUE の証拠を人が承認 → ③再生成 → 再 freeze
(c) 仕様が誤り	②①自体が間違い	ISSUE で人が裁定 → ①改訂(下流 stale は自動検知)

- ④⑤ループ上限は 3 回(attempt = 最後の裁定以降の⑤実行回数)。到達で triage ISSUE 自動起票 → ledger に blocked_by 記録 → WBS 。④⑤ skill は blocked 中の実行を拒否
- 復帰: 人が ISSUE を裁定 → AI が反映(applied) → ledger unblock → ⑤を再トリガ(attempt リセット)。ブラウザではこれが 1 操作に統合されている — 裁定待ちの関数の仕様書ページに ISSUE の質問がそのまま出て、回答を送ると「ISSUE への記入(answered) → unblock → サイト更新」まで済む。リロード後に「⑤を実行する」ボタンに切り替わる

3.3.7 ⑥ 完了検証

- 全関数完了後に 1 回。ledger check(全関数の status・ハッシュ・成果物の総点検)+ review_checks.py all(①②の全関数機械レビュー)が両方 exit 0 で pass
- docs/completion-check.md を自動生成。不備リストが出たら該当フェーズへ差し戻し
- 最終レンダリング(HTML サイト+PDF 合本)もここで実施
- LLM を使わないので数十秒で終わる。WBS トップのボタンからも実行できるが、**全関数の⑤が完了するまでボタン自体を出さない**(時期の来ていない操作を見せない)

3.3.8 ⑦ 分析・改善

移植完了後の DevOps ループ。1 施策 = 1 票 = 1 イタレーション。

1. **計測・走査**: ここは 2 段構えにしてある。「計測せずに提案しない」を仕組みで担保するため。
 - 1 段目(LLM 不使用): quant_analyze.py が cProfile + radon / ruff / bandit / pip-audit を 機械実行し、実測データを .legacy-reverse/quant.json と docs/quant.md・docs/perf.md に残す
 - 2 段目: AI がその実測データだけを読んで docs/analysis.md(OPT- / REF- / SEC- 施策台帳)に候補を記入する。**施策の適用・コード変更・施策票の起票はここではしない**

ホットスポット判断は代表ワークロード(ルートの bench.py)で行う。無い場合はテスト負荷のスモーク計測になり、その旨が計測サマリに明記される
 2. **項目確定**: 候補を施策票(docs/improvements/OPT-001.md 等)に昇格。票には目的・検証可能な成功基準(baseline → 目標)・改善仕様。人が承認
 3. **イタレーション**: 適用(1 施策 = 1 コミット) → テスト全 pass 確認 → 再計測 → 実測と判定を記録。達成なら achieved 、未達なら巻き戻して振り返り
 4. WBS の「⑦改善イタレーション」表で全施策の達成状況をトレース
- **挙動保存が絶対条件**: ③のテスト資産を安全網に「テスト全 pass 維持」で適用。挙動変更が必要なら ISSUE → ①②経由(⑦では行わない)。tests/ の編集も禁止
 - Rust(PyO3) 化のような大施策は 4 段階基準(CPU バウンドか → NumPy で足りないか → 純粋関数か → 保守コスト明記)で根拠付き提案 → 人が票で判断
 - status 遷移: proposed → approved → applied → verified(achieved / not-achieved)

4. データ仕様

iノート

スキーマの機械的な正は `legacy-reverse/references/schema.md`(skill が実行時に参照する原本)。本章はそれに設計意図を加えた解説版。両者が食い違ったら `schema.md` が正。

4.1 データ設計の原則(全体)

- **functions.json が正データ**。⑥の機械抽出が生成・マージする。WBS・骨子・Sphinx 索引はすべてここから再生成される派生物。手書き修正は `functions.json` に対して行い、派生物は作り直す
- **完了状態は成果物自身が持つ**(フロントマターの `status` / ハッシュ)。別の進捗 DB を持たない。「ファイルが正しければ進捗も正しい」が常に成り立ち、**再開が自明**になる
- **ledger.json はスクリプト専用の最小限の台帳**(③ freeze ハッシュ・blocked・attempt 基準時刻)。人も LLM も手編集しない
- 機械可読メタデータは YAML フロントマターに集約。WBS・⑥はフロントマター走査で自動生成
- 成果物フロントマターに独自キーを足すときは Quarto 予約キーと衝突させない (前例: `coverage` → `tc-coverage` に改名)
- **表示専用の使い捨てデータは `.legacy-reverse/` に隔離**する。進捗の正は常に成果物側にあり、`.legacy-reverse/` を消しても失われるのは実行ログと画面表示だけ

4.1.1 正データが壊れたときの多層防御

`functions.json` は全スクリプトが毎回読む。ここが壊れると無人バッチが途中で止まるため、AI の編集ミス(末尾カンマ等)を 3 段で受け止める。

表 4.1: `functions.json` の破損を受け止める 3 段の防御

段	仕組み	効果
1. 発生の直後	<code>hooks/guard_json.py</code> (PostToolUse)が編集直後に構文検証	壊れた位置(行・列)を添えて エージェント自身に差し戻す 。人を待たずにその場で直す
2. 読み込み時	<code>ledger.load_json</code> が行番号付きの明示エラーで停止	生の <code>traceback</code> ではなく「どこが・どう壊れ・どう直すか」を出す
3. 実行系	バッチ・単発・サーバがこれを「データ異常のため停止」として扱う	画面に理由が残る(無言で止まらない)

4.2 対象プロジェクトのディレクトリ構成

リスト 4.1: 対象プロジェクトの成果物と実行時データの配置

```
<project>/
  legacy/                # レガシー原文(読み取り専用。読めるのは ⑥ と ①改訂のみ)
```

```

src/<package>/      # ④ 新実装
tests/              # ③ テストコード(⑤ループ中は hook が編集拒否)
docs/
  _quarto.yml        # HTML サイト+PDF 出力設定(テンプレから⑩で配置)
  wbs.css             # WBS 関数一覧の列幅(テンプレからコピー)
  index.qmd           # WBS(ledger.py wbs で自動生成。手編集禁止)
  wbs/               # 200 関数超で自動生成されるファイル別明細(手編集禁止)
  spec-review.md      # ① draft の一斉レビュー表(自動生成。手編集禁止)
  review-feedback.md  # ブラウザからの修正依頼(サーバが追記・skill が applied 化)
  _site/             # HTML 出力(render_site.py が作り直す。git 管理外)
  _sitework/         # render 用の .qmd 影コピー(作業用。残らない)
  conventions.md      # プロジェクト規約(⑩で確定。人が編集可)
  domain-knowledge.md # 裁定の蓄積(人が編集可)
  specs/F-xxxx.md     # ① 関数仕様書
  test-specs/F-xxxx.md # ② テスト仕様書
  test-results/F-xxxx_日時.md # ⑤ 結果報告書(毎回新規・自動生成)
  issues/ISSUE-xxx.md # 質問と裁定(回答欄は人が記入可)
  improvements/XXX-nnn.md # ⑦ 施策票
  completion-check.md # ⑥(自動生成)
  quant.md / perf.md  # ⑦ 定量計測の結果(quant_analyze.py が生成)
  analysis.md         # ⑦ 施策候補(AI が実測に基づいて記入)
docs-sphinx/        # ④ API ドキュメント設定(テンプレから⑩で配置)
data/
  functions.json      # ⑩の解析結果(正データ)
  extract-report.json # ⑩機械抽出の監査ログ
  ledger.json         # ハッシュ・ブロック状態(スクリプト専用)
.legacy-reverse/     # 実行時データ(git 管理外・消しても成果物は失われない)
  state.json         # 実行中フェーズ(hook の判定に使う)
  run.lock           # 実行スロットの排他(PID 入り。残骸は自動解除)
  pipeline-status.json # ライブ進捗(バッチ実行状況ページが読む・表示専用)
  pipeline-log.jsonl # 実行ログ 1 行 1 試行(結果・コスト・所要)
  agent-logs/F-xxxx.txt # エージェント応答の全文(失敗原因の一次情報)
  batch-priority.json # ★優先の割り込み順(画面が書く)
  last-run.json      # pytest プラグインの出力(⑤の一時データ)
  quant.json         # ⑦の実測データ(機械可読)

```

state.json と run.lock は役割が違う。前者は hook が「いまどのフェーズか」を判定するためのフェーズ状態、後者は実行の二重起動を防ぐ排他ロック。

4.3 data/functions.json

⑩の機械抽出(extract_fortran.py / extract_c.py)が生成・マージする正データ。言語混在プロジェクトでは複数の抽出器が同じファイルにマージする。

リスト 4.2: 正データ functions.json のスキーマ

```

{
  "project": {
    "name": "string",
    "legacy_lang": "fortran | csharp | ...",
    "new_lang": "python",
    "package": "newpkg"
  },
  "functions": [
    {
      "func_id": "F-0123",
      "legacy": { "file": "legacy/tax.cbl", "name": "CALC-TAX", "lines": "120-240" },
      "new": {
        "module": "src/newpkg/tax.py",

```

```
      "name": "calc_tax",
      "signature": "calc_tax(amount: Decimal, rate_code: str) → Decimal"
    },
    "inputs": [ { "name": "WK-AMOUNT", "legacy_type": "PIC 9(9)V99", "new_type":
"Decimal", "desc": "課税対象額" } ],
    "outputs": [ { "name": "WK-TAX", "legacy_type": "PIC 9(9)V99", "new_type":
"Decimal", "desc": "税額" } ],
    "globals": [ { "name": "TAX-RATE-TABLE", "access": "read", "desc": "税率マスタ" } ],
    "external_files": [ { "path": "RATEMST.dat", "access": "read", "desc": "税率マス
タファイル" } ],
    "calls": [ "F-0087" ],
    "test_file": "tests/test_tax.py"
  }
]
```

- func_id は F- + 4 桁連番。採番は抽出器が行い、マージ再実行でも不変
- F-0000 はメインルーチンの予約番号(Fortran の program / C の main)。コールグラフの根なの
で推奨着手順では最後に回る
- calls は func_id の配列 — コールグラフの正データ(WBS の推奨着手順=葉から、の根拠)
- unresolved_calls(任意)は抽出時に解決できなかった呼び出し名。別言語の抽出が 走った時点で自動的に解決されて calls に移るので、抽出の実行順を問わない
- test_file は④では省略可。③が確定させる

4.3.1 人が後から調整するためのフラグ

④の抽出漏れ・関数分割・デッドコードは、人の指示で後追い調整する (ledger add / exclude / include、MCP では add_function 等)。

表 4.2: 人が後追いで付ける調整フラグと設計判断

フラグ	意味	設計判断
manual: true	人の指示で後追い追加したエントリ	抽出の再実行で「ソースに無い」警告を出さない。実ソースで確認されたら自動で外れる
excluded: true + excluded_reason	移植対象外	①～⑥・WBS・next の対象から外れ、WBS の「対象外の関数」に理由つきで残る

エントリの物理削除は禁止。抽出を再実行するとソースから別の func_id として復活し、既存の成果物との紐付けが切れるため。対象から外すのは必ずフラグで行う。

4.3.2 data/extract-report.json(監査ログ)

④の機械抽出のたびに更新される。人と LLM がレビューすべき項目:

表 4.3: 抽出の監査ログでレビューすべき項目

項目	意味
completeness_mismatch	② 系統カウント突合の差分(数え漏れの疑い)。ゼロが④完了条件
inferred_calls	静的解析で推定した呼び出し関係
unresolved_calls	解決できなかった呼び出し名(外部ライブラリ等の可能性)

項目	意味
マージ差分	再実行時の追加・変更・保持の内訳

4.4 data/ledger.json

スクリプト(ledger.py / collect_results.py)だけが読み書きする。手編集禁止。

リスト 4.3: スクリプト専用台帳 ledger.json のスキーマ

```
{
  "F-0123": {
    "test_code_hash": "e5f6a7b8",
    "blocked_by": null,
    "attempt_reset_at": "2026-07-25T10:00"
  }
}
```

表 4.4: 台帳の各キーが保持する状態

キー	意味
test_code_hash	③ freeze 時のテストファイル sha256 先頭 8 桁。現物と不一致 = ⚠️ 改変
blocked_by	④⑤ ループ上限到達時の ISSUE-ID。人の裁定後 <code>unlock</code> で null に戻る
attempt_reset_at	この時刻以降の結果ファイル数 = attempt(⑤ 実行回数)。unlock でリセット

4.5 .legacy-reverse/state.json

リスト 4.4: hook が参照するフェーズ状態のスキーマ

```
{ "phase": "5", "func_id": "F-0123" }
```

- フェーズ skill が開始時に `ledger.py phase-start <n> <func-id>`、終了時に `phase-end` で管理
- hook(guard_tests.py)は phase が 4 または 5 のとき tests/ への Edit/Write を拒否する

4.6 .legacy-reverse/pipeline-status.json

実行中の状態。**表示専用の使い捨て**で、進捗の正ではない(正は成果物側)。書き込みはアトミック(tmp → replace)なので、読み手が壊れた JSON を掴むことはない。

リスト 4.5: ライブ進捗 pipeline-status.json のスキーマ

```
{
  "state": "running",
  "mode": "連続実行(ブラウザ)",
  "pid": 12345,
  "total_targets": 62, "done": 34, "failed": 5,
  "cost_usd": 11.83,
  "rate_waited_sec": 340, "wait_until": null,
}
```



```

"current": { "func_id": "F-0032", "attempt": 1, "started_at": "2026-08-07T06:12:03" },
"ng_kinds": { "機械レビュー NG": 3, "検証 NG": 2 },
"recent": [
  { "func_id": "F-0019", "ok": false, "phase": "spec",
    "kind": "機械レビュー NG", "problems": ["SPEC-...: 根拠引用がない"],
    "sec": 233, "cost_usd": 0.66, "attempt": 2, "at": "2026-08-07T06:03:28" }
],
"metrics": { "median_sec": 112, "success_rate": 0.872,
  "eta_sec": 2576, "avg_cost_usd": 0.303 }
}

```

表 4.5: 画面表示が依存する主要キーとその役割

キー	意味
state	running / waiting_rate / stopped / finished / not_running
pid	書いたプロセス。死んでいれば画面側が「異常終了」に読み替える (残骸で永久に実行中に見えるのを防ぐ)
recent[].phase	どの工程だったか(spec / testspec / …)。連続実行は工程横断なので、 これが無いと結果行が何の失敗か分からない
recent[].problems	機械レビューの指摘の全文。画面で箇条書きにする(件数だけ出して 探しに行かせない)
metrics.median_sec	成功した実行の所要時間の中央値。画面の進捗バーの基準になる

4.7 .legacy-reverse/batch-priority.json

画面の「★優先」が書く割り込み指示。スクリプト専用。

リスト 4.6: ★優先の割り込み指示 batch-priority.json のスキーマ

```

{ "order": ["F-0033", "F-0021"], "retry": ["F-0021"] }

```

表 4.6: 割り込み指示のキーと消費の規則

キー	意味	消費
order	実行順の割り込み。この順で走査結果の先頭に並べ替える	残り続ける(★解除まで)
retry	失敗スキップ集合からの解除対象	次の走査で消費されるワンショット

retry をワンショットにするのは、毎回解除すると失敗し続ける関数が無限に再試行されてバッチが進まなくなるため。

4.8 フロントマターの status 遷移(完了判定の正)

WBS の ☒ 判定・ledger next の着手可否判定は、すべてこの表に従う(判定ロジックは ledger.py の status_of() に一元化)。

表 4.7: 成果物ごとの status 遷移と WBS の完了条件

成果物	遷移	WBS ☒ 条件
① spec	skeleton → draft → reviewed	reviewed
② test-spec	generated → approved	approved かつ spec-hash が現物と一致
③ test-code	(ファイル+ledger)	ledger の test_code_hash が現物と一致
④ impl	(ファイル)	new.module が存在しスタブ検出ゼロ
⑤ test	(最新結果ファイル)	result: pass(= 実装率 100% かつ失敗 0)
ISSUE	open → answered → applied	— (open は WBS 最上部に露出)
⑦ 施策票	proposed → approved → applied → verified	achieved

4.8.1 ハッシュ連鎖の実装

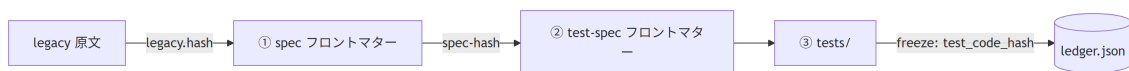


図 4.1: レガシー原文から台帳まで伝搬するハッシュ連鎖

- ハッシュは `ledger.py hash <path>`(sha256 先頭 8 桁)
- `ledger verify <func-id>` が連鎖を検証。不一致 = 上流が改訂された = 下流は「要再生成」
- ⑤の実行前に必ず `verify` が走るため、stale なテストで pass を出すことはできない

4.9 WBS の表示記号

表 4.8: 関数一覧に出る記号と対応する状態

記号	意味
☒	完了(上表の条件を満たす)
▲	作業中(draft 等)
□	未着手
⊖	裁定待ち(blocked。ISSUE に回答を)
⚠	stale(上流改訂)または改変(freeze 後変更)→ 要再確認

5. 品質ゲート仕様

5.1 全体像

LLM の成果物は、人のレビューに届く前に**決定的スクリプトの検証**を必ず通る。「NG が残る成果物を『できました』と報告してはいけない」が全フェーズ skill の共通規則。

表 5.1: フェーズごとの品質ゲートと検知対象

フェーズ	ゲート	検知対象
① 完全性	抽出器(extract_fortran.py / extract_c.py)内蔵	2 系統カウンタ突合による数え漏れ
① draft 後	review_checks.py spec	実在しない <code>file:lines</code> 引用、● なのに根拠なし、プレースホルダ残存(省略)、必須節欠落、原本ハッシュ不一致
② 承認依頼前	review_checks.py testspec	● 仕様項目のケース漏れ、①に無い SPEC-ID 参照(捏造)、宙に浮いた TC 参照、根拠の規定外表記、spec-hash 鮮度
③ freeze 前	pytest -collect-only + マーカー突合	ケース ID とテスト関数の過不足 (collect_results が exit 3)
④ 完了前	check_stubs.py	空実装・NotImplementedError・TODO/FIXME(スタブ化)
⑤ 事前	ledger verify	② stale・freeze 後のテスト改変・blocked の見落とし
⑥ 前	review_checks.py all	全関数の①②総点検
承認の瞬間	サーバ側で機械レビューを再実行	クライアント表示の改竄・競合による「NG 付き成果物の承認」
全編集の直後	hooks/guard_json.py	AI の編集による正データ(JSON)の破損

この結果、人のレビューは「意味が正しいか」に集中できる。「引用された行は実在するか」「トレーサビリティは揃っているか」の照合作業は機械が済ませている。

5.2 機械で取れないもの、の分担

機械検知の限界は「式は書いてあるがレガシーの意図と違う」という**意味レベルのすり替え**。これは次の 2 つが受け持つ:

1. 人の①レビュー — Confidence(●●●)と行番号根拠が付いた状態で意味を確認する
2. ⑤の突き合わせ — 独立に作られたテスト(②③系)と実装(④系)の不一致として露見する

疑わしければ ISSUE(仮説+Yes/No)へ。これが「機械が正、AI は意味づけ、裁定は人」の分担。

5.3 review_checks.py の検査仕様

5.3.1 ① spec 検査(review_checks.py spec <fid>)

表 5.2: ①仕様書の検査項目と検知するハルシネーション

検査	検知するハルシネーション/手抜き
根拠引用の实在	file:lines が実在するレガシー行を指しているか(存在しない行への引用 = 捏造)
●の根拠必須	Confidence ● (確認済)なのに根拠引用がない項目
プレースホルダ残存	テンプレの穴埋めが残っている(記入の省略)
必須節の欠落	テンプレ(assets/templates/spec.md)が定める節の欠落
原本ハッシュ鮮度	仕様書が参照するレガシー原本が書き換わっていないか

5.3.2 ② testspec 検査(review_checks.py testspec <fid>)

表 5.3: ②テスト仕様書の検査項目とトレーサビリティ検証

検査	検知するもの
トレーサビリティ(下り)	①の●仕様項目すべてにテストケースが対応しているか(ケース漏れ)
トレーサビリティ(上り)	参照している SPEC-ID が①に実在するか(仕様の捏造検知)
TC 参照の整合	宙に浮いたケース参照がないか
期待値根拠	根拠の表記が規定(仕様●/人間確認/実測)に準拠しているか
⚠未確定	未確定件数(approved の前提はゼロ)
spec-hash 鮮度	①改訂後の古い②でないか

5.3.3 一斉レビュー支援(review_checks.py report)

① draft の一斉レビュー表 docs/spec-review.md を生成(バッチ運用の要)。概要・Confidence 内訳・未回答質問を関数ごとに並べ、人が「全部 OK / F-xxxx は修正」で返せる。

品質ゲートの観点で重要なのは、この表が機械レビューの結果を人の作業順に翻訳していること。

表 5.4: 一斉レビュー表の仕掛けと品質上の意味

表の仕掛け	品質上の意味
「承認できる → ISSUE あり → AI 修正待ち」の順に並べる	人が判断できる行から消化でき、判断のいない行を眺めさせない
✖の行には承認ボタンを出さず「AI 修正待ち」と表示	機械 NG は人が承認できないことを UI で担保する(押せてしまう余地を残さない)
✖の件数リンクが仕様書ページの理由全文へ飛ぶ	「4 件 NG」だけ見せて中身を探させない

表示の詳細は 画面設計 ― 一斉レビュー表。

5.3.4 総点検(review_checks.py all)

全関数の①②を一括検査(skeleton は除外)。⑥前の総点検と再開時の健全性確認に使う。

5.4 承認経路の強制(サーバ側の再検証)

承認は 2 つの UI(仕様書ページの承認ウィジェット・一斉レビュー表の行内ボタン)から 行えるが、**検証と記録は 1 本**にしてある。

- 承認 POST を受けた時点で**サーバ側が機械レビューを再実行**し、NG なら承認を拒否する。ボタンの活性制御(グレーアウト)は補助であって、それだけには依存しない
- status が期待値でない(既に承認済み・他で更新された)場合も拒否する。複数の画面・複数の人が同時に触っても、状態が壊れない
- 承認の記録先はフロントマターの reviewed-by / reviewed-date(②は approved-*)で、チャット承認・ブラウザ承認のどちらでも同じフィールドに書く

5.5 hooks: 拒否と差し戻しの 2 系統

LLM の自制に頼らず、**仕組み**で強制する層。PreToolUse は「やらせない」、PostToolUse は「やった結果を検査して直させる」という違いがある。

表 5.5: 拒否系と差し戻し系の 2 つの hook

hook	種別	何をするか
guard_tests.py	PreToolUse	phase が 4 / 5 のとき tests/ への Edit/Write を 拒否 する
guard_json.py	PostToolUse	.json の編集直後に構文検証し、壊れていれば エージェントに差し戻す

5.5.1 guard_tests.py(テスト改変の物理防止)

「AI がテストを書き換えて通す」事故を防ぐ。

- .legacy-reverse/state.json の phase が 4 または 5 のとき、tests/ への Edit/Write を拒否
- フェーズ skill は開始時 ledger phase-start、終了時 phase-end で状態を明示する契約
- テスト側に疑義があるときの正規経路は ISSUE → 人承認 → ②③再生成(hook の迂回ではない)

5.5.2 guard_json.py(正データ破損の防止)

data/functions.json は全スクリプトが毎回読む正データで、ここが壊れると 無人バッチが途中で止まる。編集の直後に json.loads で検証し、失敗したら **壊れた位置(行・列)と直し方を添えて exit 2 で差し戻す**。

- 「拒否」ではなく「差し戻し」なのは、**JSON の編集自体は正当な作業**だから。禁じるべきは編集ではなく、壊れたまま次に進むこと
- エージェント自身がその場で直すので、人の復旧作業を待たない
- すり抜けた場合の受け皿は データ仕様 ― 正データが壊れたときの多層防御

hook は MCP 化しない(ツール呼び出しの強制ガードは hook の役割のまま)。

5.6 スタブ検知: check_stubs.py

④の「動いた風」を防ぐ。検知対象:

- 本体が pass / ... だけの関数(空実装)

- `NotImplementedError`
- TODO / FIXME コメント

検知ありは④未完了扱い。実装しきれない場合の正規経路は spec-gap ISSUE → ①改訂。

5.7 ⑤の制御信号(`collect_results.py` の exit code)

表 5.6: `collect_results.py` の exit code と後続処理

exit	result	意味	後続
0	pass	実装率 100% かつ失敗 0	WBS 
1	fail	失敗あり	トリアージ((a) は自走、attempt +1)
2	blocked	attempt 上限 3 到達	triage ISSUE 自動起票・WBS  ・人の裁定待ち
3	mismatch	②のケース ID と③のマーカー不整合	③の突合修正(freeze 前なら③へ差し戻し)

5.8 完了検証: `ledger check`(⑥)

フロントマター走査+ハッシュ照合で全関数を総点検し `docs/completion-check.md` を生成。

- ①～⑤の完了条件(データ仕様 — status 遷移)を全関数で検査
- open ISSUE・blocked・stale・改変の残存を列挙
- `review_checks.py all` と併せて両方 exit 0 が⑥ pass

6. 画面設計

i この章の範囲と「正」

人が触れる画面(HTML)と、その裏側の HTTP エンドポイント・状態ファイル・描画タイミングを扱う。「どの画面に何が出るか」「その表示は誰が・いつ作るか」が主題。

挙動の正は `references/workflow.md`(機械が従う実行時規則)、操作手順の正は `MANUAL.md`(操作者向け)にあり、本章はそこから設計意図を解説する。フェーズそのものの仕様はパイプライン仕様、データの形はデータ仕様を参照。

6.1 画面が担うもの

このシステムの人の仕事は4つ — トリガを引く・質問に答える・承認する・裁定する。画面はこの4つを「チャットに戻らずに」完結させるために存在する。逆に言えば、画面は進捗の閲覧装置ではなく操作装置として設計されている。

表 6.1: 画面ごとの URL と役割、誰が生成するか

画面	URL	主な役割	生成方法
WBS(トップ)	/	進捗の全体像・要対応・次の一手。⑥⑦の起動	Quarto(<code>ledger.py</code> が <code>index.qmd</code> を生成)
関数仕様書	/specs/F-xxxx.html	①の閲覧。関数の「ホーム」として①～⑤の実行・承認・裁定ボタンが集まる	Quarto(成果物 <code>.md</code> + ウィジェット焼き込み)
① 一斉レビュー	/spec-review.html	draft の仕様書をまとめて承認・修正依頼	Quarto(<code>review_checks.py</code> が <code>report</code> が <code>.md</code> を生成)
バッチ実行状況	/pipeline.html	無人実行の運転席。状況把握・順番の割り込み・人待ちの解消	<code>serve_site.py</code> が動的配信(Quarto を通さない)
⑥ 完了検証	/completion-check.html	全関数の完了判定レポート	Quarto(<code>ledger.py</code> が <code>check</code> が生成)
⑦ 分析 / 定量評価	/analysis.html • /quant.html	施策候補と実測データ	Quarto
新コード詳細(API)	/api/index.html	④実装の docstring から生成した API リファレンス	Sphinx(別系統)

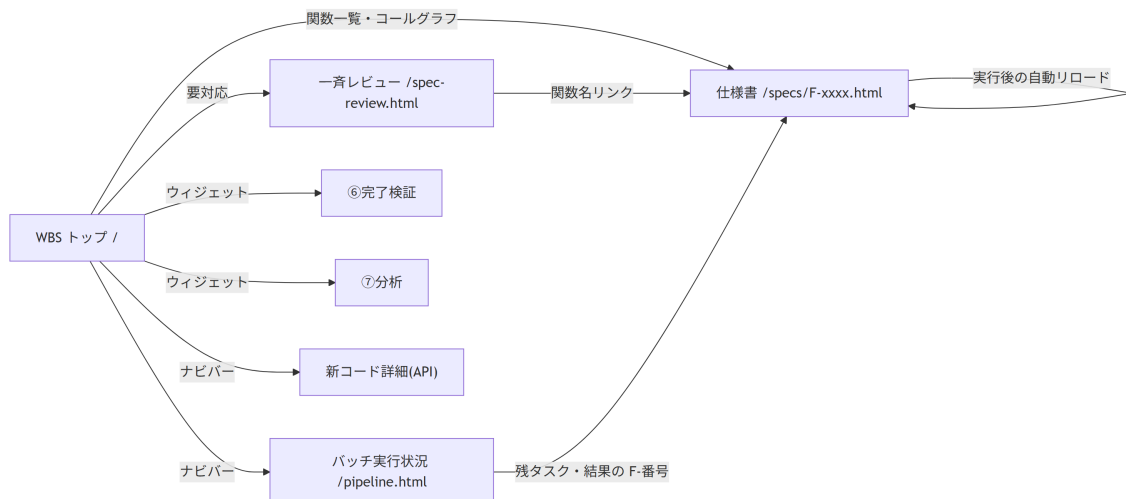


図 6.1: 画面遷移と、どこから何へ行けるか

6.2 配信アーキテクチャ

6.2.1 静的配信 + 3 つの動的経路

`serve_site.py` は Quarto が出力した静的サイト(`docs/_site`)を配る HTTP サーバで、そこに動的な経路を 3 種類だけ足している。

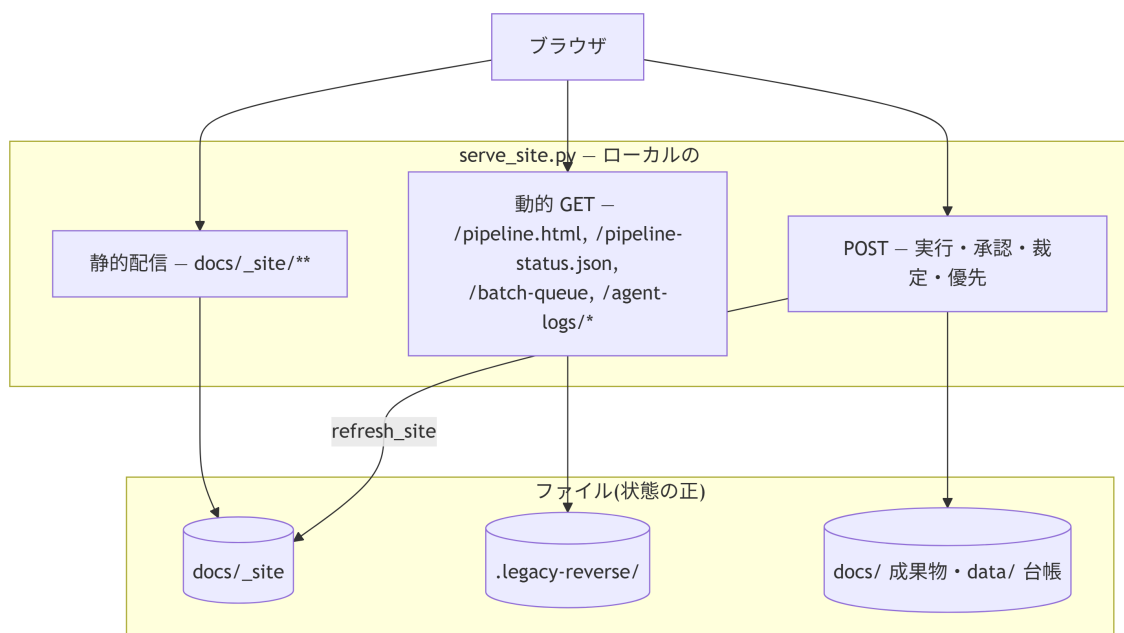


図 6.2: 静的配信に 3 つの動的経路を足した構成

なぜ全部を Quarto に通さないか。成果物のページ(仕様書・WBS・一斉レビュー)は Markdown を正とし Quarto でレンダリングするが、バッチ実行状況ページだけは `serve_site.py` に埋め込んだ HTML 文字列(`PIPELINE_PAGE`)を直接配る。理由は 2 つ。

- **更新頻度:** 実行中は秒単位で変わる。1 回の再レンダリングに数十秒かかる Quarto を経由すると、そもそも「ライブ」にならない
- **成果物ではない:** 実行状況は使い捨ての運転情報であって、リポジトリに残す文書ではない。Markdown を正とする必要がない

逆に一斉レビュー表は「レビュー対象の一覧」という**成果物**なので Markdown を正とし、操作性(フィルタ・検索・行内ボタン)はページ内 JS で後付けする、という分担にしている。

6.2.2 サーバを挟む理由(素の http.server を使わない)

表 6.2: 素の http.server を使わない 6 つの理由

理由	実装
プロジェクトごとに固定ポート	プロジェクト名の CRC32 から 8100-8899 を決定。複数プロジェクトが衝突しない
ローカル限定	既定で 127.0.0.1 にのみ bind(仕様書は社内資料)。LAN 公開は明示オプション+警告
キャッシュ無効	Cache-Control: no-store。再レンダリング後にリロードだけで最新が出る
MIME の確定	Windows のレジストリ由来 MIME は当てにならない(.js が text/plain になり Mermaid が動かない)ため上書き
書き込み経路の提供	承認・実行・裁定の POST(後述の安全設計つき)
EXE 配布	PyInstaller のエントリポイント。_site を同梱して単体実行ファイル化できる

6.3 HTTP エンドポイント仕様

6.3.1 GET

表 6.3: GET 経路が返すものと、その実装

パス	返すもの	実装
/pipeline • /pipeline.html	バッチ実行状況ページ(HTML 定数)	PIPELINE_PAGE
/pipeline-status.json	ライブ進捗。クラッシュ残骸の補正つき	_mark_stale_status
/batch-queue?q=&chip=	残タスク一覧(実行順・検索・件数集計)	browser_run.batch_queue
/agent-logs/<fid>.txt	エージェント応答の全文	ファイル名を [\\w\\.\\-]+\\.txt で検証してから配る
/favicon.ico	無ければ 204	404 でログを汚さないため
その他	docs/_site 配下の静的ファイル	SimpleHTTPRequestHandler

_mark_stale_status は、状態ファイルが running のまま残っている(実行プロセスが異常終了した)場合に、PID の死活を確認して stopped + 理由に読み替えて配る。ファイル自体は書き換えないう読み取り専用の補正で、これが無いと画面だけが永遠に「実行中」に見える。

6.3.2 POST(すべて書き込み・実行系)

表 6.4: POST 経路 8 種の用途と、その実装

パス	用途	実装
/review-action	①②の承認 / 修正依頼、⑤の裁定	<code>review_actions.approve / request_changes / adjudicate</code>
/run-phase	①～⑤の単発実行	<code>browser_run.start</code>
/run-batch	連続実行の開始	<code>browser_run.batch_start</code>
/run-cancel	実行の中止(単発・連続とも)	<code>browser_run.cancel</code>
/run-skip	連続実行の「いまの 1 件」だけスキップ(バッチは続行)	<code>browser_run.skip_current</code>
/batch-priority	★優先の ON/OFF	<code>browser_run.prioritize</code>
/run-check	⑥完了検証(LLM 不使用・同期)	<code>browser_run.run_check</code>
/run-analyze	⑦分析の開始	<code>browser_run.analyze_start</code>

応答は常に `{"ok": bool, "message": str}`。 `ok: false` は HTTP 422 で返し、**message はそのまま画面に出す前提の日本語**にする(「なぜできないか」が画面上で完結するように)。

6.3.3 安全設計

書き込み経路は次の 4 段で守る。

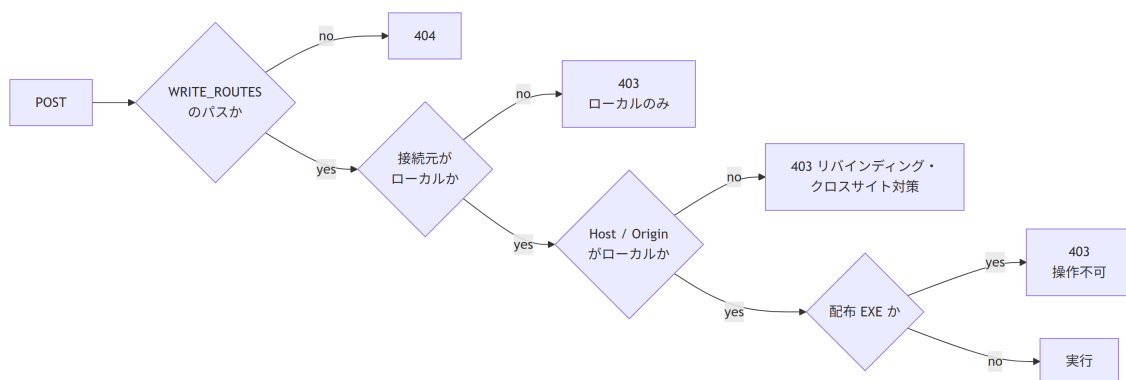


図 6.3: 書き込み POST を守る 4 段のガード

- **接続元 IP の確認だけでは足りない。**(a) 攻撃者ドメインの DNS を `127.0.0.1` に向けて同一オリジンから fetch させる DNS リバインディング、(b) `text/plain` フォームで preflight を回避して JSON を送り込むクロスサイト POST、の 2 経路が残る。`Host` と `Origin` のホスト名までローカルであることを確認して塞ぐ
- `--host 0.0.0.0` で LAN 公開しても書き込みはローカル限定。閲覧は共有できるが、リモートから成果物を書き換え・実行させない
- **配布 EXE(FROZEN)では全 POST を 403。**EXE の中身はビルド時点のスナップショットで、操作しても元プロジェクトに反映されないため、できないことを明示して返す
- **承認はサーバ側で機械レビューを再検証してから確定する。**クライアントの表示 (ボタンが押せた=OK) を信用しない。「NG 付きの成果物を承認しない」という原則を UI の見た目ではなくサーバで強制する

6.4 ウィジェット — 状態に応じて画面に現れる操作 UI

6.4.1 2つの実装方式

表 6.5: ウィジェット 2 方式と、更新のされ方の違い

方式	対象	更新のされ方
レンダ時に焼き込み	仕様書ページの 3 ウィジェット、WBS の⑥⑦ボタン、一斉レビュー表の行内ボタン	状態が変わったら再レンダリングが必要。操作後に <code>refresh_site</code> が自動で走る
ページ内で動的生成	バッチ実行状況ページの全要素	JSON ポーリングで秒単位に更新

焼き込み方式には「操作した瞬間には画面が変わらない」という制約があるため、**POST の成功後に `refresh_site`(WBS 再生成 + 一斉レビュー表再生成 + 差分レンダ)をサーバ側で実行し、完了後にページをリロード**する。この順序が重要で、先に「完了」を返すとポーリング側が古いページのままりロードしてしまう。

JS の実体は `browser_run.WIDGETS_JS` の 1 ファイルに集約し、`render_site.py` がレンダ後に `_site/lr-widgets.js` として書き出す。各ウィジェットの HTML は `<script src="/lr-widgets.js">` で参照する(ページごとに JS を複製しない)。公開オブジェクトは `lrRun / lrReview / lrAdj / lrCheck / lrAnalyze`。

6.4.2 出現条件の決定表

仕様書ページ(`/specs/F-xxxx.html`)は関数の「ホーム」として、そのときに可能な操作を **必ず 1 つ** だけ出す。二重導線を作らないため、承認待ちの間は実行ボタンを出さない。

表 6.6: 関数の状態ごとに出るウィジェットは高々 1 つ

関数の状態	出るウィジェット	実装
① skeleton	「①を実行する」	<code>trigger_widget_html</code>
① draft(指摘なし)	承認ウィジェット(承認 / 修正依頼)	<code>widget_html(kind=spec)</code>
① draft(機械レビュー NG or 修正依頼あり)	承認ウィジェット + 再実行ボタン	同上 + <code>_rerun_wanted</code>
① reviewed ・ ② なし	「②を実行する」	<code>trigger_widget_html</code>
② generated	承認ウィジェット(テスト仕様書)	<code>widget_html(kind=testspec)</code>
② approved ・ ③ 未 freeze	「③を実行する」	<code>trigger_widget_html</code>
③ freeze 済み ・ ④ 未実装	「④を実行する」	<code>trigger_widget_html</code>
④ 実装済み ・ ⑤ 未 pass	「⑤を実行する」	<code>trigger_widget_html</code>
⑤ blocked(🔴)	裁定ウィジェット(ISSUE の質問を表示・回答して unblock)	<code>adjudicate_widget_html</code>
完了	何も出ない	—

WBS トップには2つ出る。どちらも**時期が来るまで表示しない**(押せないボタンを置かない)。

表 6.7: WBS トップの⑥⑦ボタンが現れる条件

条件	ウィジェット
全関数の⑤が完了	「⑥完了検証を実行する」(LLM 不使用・数十秒で同期完了)
⑥が pass	「⑦分析を実行する」(定量計測 → 施策候補の提案まで。コードには触らない)

仕様書ページは3種のウィジェットの候補を持つが、**出すのは高々1つ**で、優先順は「承認 → 実行 → 裁定」。承認待ちの間に実行ボタンを並べると、「承認すべきか作り直すべきか」が画面から読めなくなるため。

6.4.3 承認ウィジェットの構造

承認ウィジェットは4つの要素で構成する。

表 6.8: 承認ウィジェット4要素と、その設計意図

要素	内容	設計意図
機械レビュー結果	✅ 問題なし / ❌ 理由の全文(箇条書き)	件数だけ出して別ページを探させない
承認する	NGの間はグレイアウト。押してもサーバ側で再検証して弾く	NG 付き成果物を承認させない
修正依頼…	<code>review-feedback.md</code> に <code>pending</code> として追記	次に AI が①②を実行したとき自動で拾われる
再実行	NG か修正依頼が残るときだけ表示	押しても空振りするボタンを置かない

理由の全文をその場に置くのが要点。「一斉レビュー表に ❌ 4 件と出ているが中身が分からない」ため別ページを探しに行く、という往復を無くしている。

6.5 画面別仕様

6.5.1 WBS(トップ)

`ledger.py wbs` が `docs/index.qmd` を生成する(手編集禁止)。200 関数を超えると自動でダッシュボードに切り替わるのが設計上の要点。

表 6.9: 200 関数を境に切り替わるトップの構成

規模	トップページの構成
200 関数以下	進捗サマリ + コールグラフ + Open ISSUE + 全関数一覧
200 関数超	進捗サマリ + 要対応 + あなたへの質問 + 次の一手 + レガシーファイル別の進捗表。全関数の明細は <code>docs/wbs/*.qmd</code> に分割

分割の理由は、2000 行の表を 1 ページに置くと「いま何が問題か」が読めなくなるため。ダッシュボード側には**判断に必要な情報だけ**(●・△・✕ と次の一手)を残す。

6.5.2 ① 一斉レビュー表

`review_checks.py` の `make_report()` が `docs/spec-review.md` を生成する。対象は `status: draft` の仕様書のみ。
並び順が優先度という設計にしている。

表 6.10: 一斉レビュー表は並び順そのものが優先度

並び	意味	操作列
1	そのまま承認できる	承認 / 修正依頼…
2	承認できるが ISSUE(人への質問)がある	承認 / 修正依頼… + ISSUE リンク
3	機械レビュー NG(AI 修正待ち)	ボタンなし(「AI 修正待ち」と表示)

- 列は 6 つ — 関数 / 概要 / Confidence / 機械レビュー / 未確定(ISSUE) / 操作。Confidence は ●●● を 3 列に分けず「2●1●」の 1 列に統合した(列数を減らして 概要に幅を回すため)。
- ・ 列幅は `table-layout: fixed` で固定する。Quarto 既定の内容比による自動配分だと、概要列が幅を占有して関数名が 1 文字ずつ縦に潰れる。余り幅は概要列だけに割り当てる
 - ・ 表に `min-width` を持たせ、画面が狭いときはフィルタバーを固定したまま**表だけ横スクロール**する
 - ・ フィルタチップ(すべて / 承認できる N / ISSUE あり N / AI 修正待ち N)と検索は ページ内 `JS(SPEC_REVIEW_PAGE_JS)` が表の直前に注入する。件数がそのまま 残作業量の指標になる
 - ・ 承認は `lrReview.approve` — **仕様書ページの承認ウィジェットと同じ経路**を呼ぶ。UI が 2 つあってもサーバ側の検証・記録は 1 本

6.5.3 バッチ実行状況ページ

無人実行の運転席。 `serve_site.py` の `PIPELINE_PAGE` が実体で、Quarto を通さない。

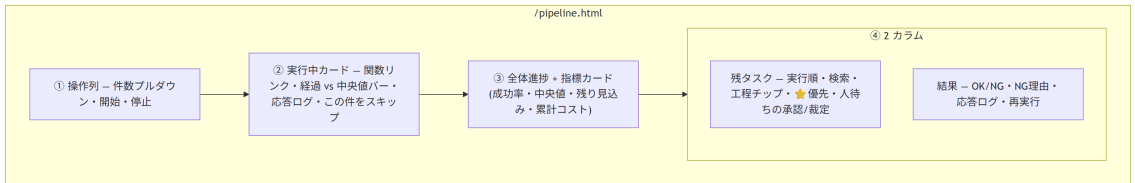


図 6.4: バッチ実行状況ページの 4 ブロック構成

6.5.3.1 設計の要点

- 「上限件数」を数値入力からプルダウンにした。数値+プレースホルダでは何を 入れるべきか伝わらない。選択肢(全部(残り N 件) / お試しに 1 件だけ / 10 / 50 / 100) そのものが説明になり、「全部」に残件数が出ることで 1 日の作業量も見積もれる。
- 予算(\$)の上限入力は UI から外した。CLI の `--budget-usd` は残してあるが、画面の操作としては件数指定で足りる(コストは指標カードで見える)。
- 実行中カードの進捗バーは「その工程の中央値」を 100% とする。絶対時間では速い/遅いの判断ができない。中央値超過で黄、2 倍超過で赤+「長引いています」と表示し、**異常への気づき**を色で伝える。
- 残タスクは既定で先頭 9 件しか出さない。2000 関数規模では全件表示が破綻するため、「次に何が来るか」だけを既定表示にし、それ以外は検索・チップで呼び出す設計にした。検索はサーバ側(`/batch-queue`)で行い、応答は 50 件で打ち切る(全件をブラウザに送らない)。

実行順の通し番号(今 / 次 1 / 次 2...)はサーバ側で振る。★優先で実行中の関数より前に割り込んだ行が「次 0」になるなどのズレを防ぐため、クライアントでの補正はしない。

人待ち(承認・裁定)を同じ画面に置く。連続実行は承認待ち・裁定待ちをスキップし続けるので、人待ちこそが実際のボトルネックになる。バッチを回しながらこの画面だけで詰まりを解消できるようにした。

6.5.3.2 残タスクの分類

`batch_queue()` は全関数を走査して、次の 2 系統に分類する。

表 6.11: 残タスクを自動実行待ちと人待ちに分ける判定

種別	kind	判定
自動実行待ち	<code>spec / testspec / testcode / impl / test</code>	<code>_decide_kind(include_rerun=True)</code> が返す工程
人待ち	<code>approve-spec</code>	① が draft(かつ再実行対象でない)
人待ち	<code>approve-testspec</code>	② が generated
人待ち	<code>adjudicate</code>	<code>ledger.blocked_by</code> あり

自動実行待ちの並びは `_scan_targets` と同じ規則(★優先 → `functions.json` 順)で、画面の並び = 実際の実行順になるようにしている。

6.5.3.3 ポーリング設計

表 6.12: 2 種のポーリング間隔と、そう決めた理由

データ	間隔	理由
<code>/pipeline-status.json</code>	2.5 秒	小さい JSON。ライブ感が要る
<code>/batch-queue</code>	約 15 秒 + 検索時・操作後	全関数の frontmatter を読むので重い。ポーリングに載せない

結果パネルは内容が変わったときだけ再描画する。2.5 秒ごとに `innerHTML` を丸ごと差し替えると、人が読もうと開いた `<details>` が毎回閉じて「チカチカしてすぐ閉じる」状態になる(報告された不具合)。差分がある場合も、開いていた行は `data-rid` で復元する。

6.5.4 配布形態による機能差

`build_viewer.py` は `_site` を同梱した単体実行 EXE を作る(レビュアーへの配布用)。このとき画面は閲覧専用になる。

表 6.13: 配布 EXE で閲覧だけになる機能の内訳

機能	<code>serve_site.py</code> で配信	配布 EXE
成果物・WBS・一斉レビュー表の閲覧	○	○
残タスク一覧(<code>/batch-queue</code>)	○	×
承認・修正依頼・裁定・実行(POST 全般)	○	× (403 と理由を返す)

EXE の中身はビルド時点のスナップショットで、操作しても元プロジェクトには反映されない。
黙って失敗させず、理由を返して「できないこと」を明示する方針にしている。

6.6 状態ファイルと排他

画面が読み書きする実行時データは `.legacy-reverse/` に置く (git 管理外・消しても成果物は失われない)。

表 6.14: 実行時状態ファイルの、書く人と読む人

ファイル	書く人	読む人	形
pipeline-status.json	RunStatus(アトミック書き込み: tmp→replace)	/pipeline-status.json	状態・件数・指標・recent[]
batch-priority.json	/batch-priority	_scan_targets	{order: [fid...], retry: [fid...]}
agent-logs/F-xxxx.txt	save_agent_log(追記)	/agent-logs/*	エージェント応答の全文
pipeline-log.jsonl	log_line	人・分析	1 行 1 試行
run.lock	ロック取得時	二重起動の判定	PID 入り

`recent[]` には工程(phase)を記録する。これが無いと結果パネルで「F-0019 が NG」までしか出せず、①なのか⑤なのかが分からない。

★優先の `retry` はワンショット。`order` は実行順の割り込みだが、`retry` は「失敗スキップ集合からの解除」で、次の走査で消費される。毎回解除にすると、失敗し続ける関数が無限にリトライされてバッチが進まなくなる。

6.6.1 実行枠の排他

実行の入口は 3 つ(チャットの skill / CLI の無人バッチ / ブラウザ)あるが、同時に走ってよい実行は 1 つだけ。どの入口からでも同じロックを取り合う。

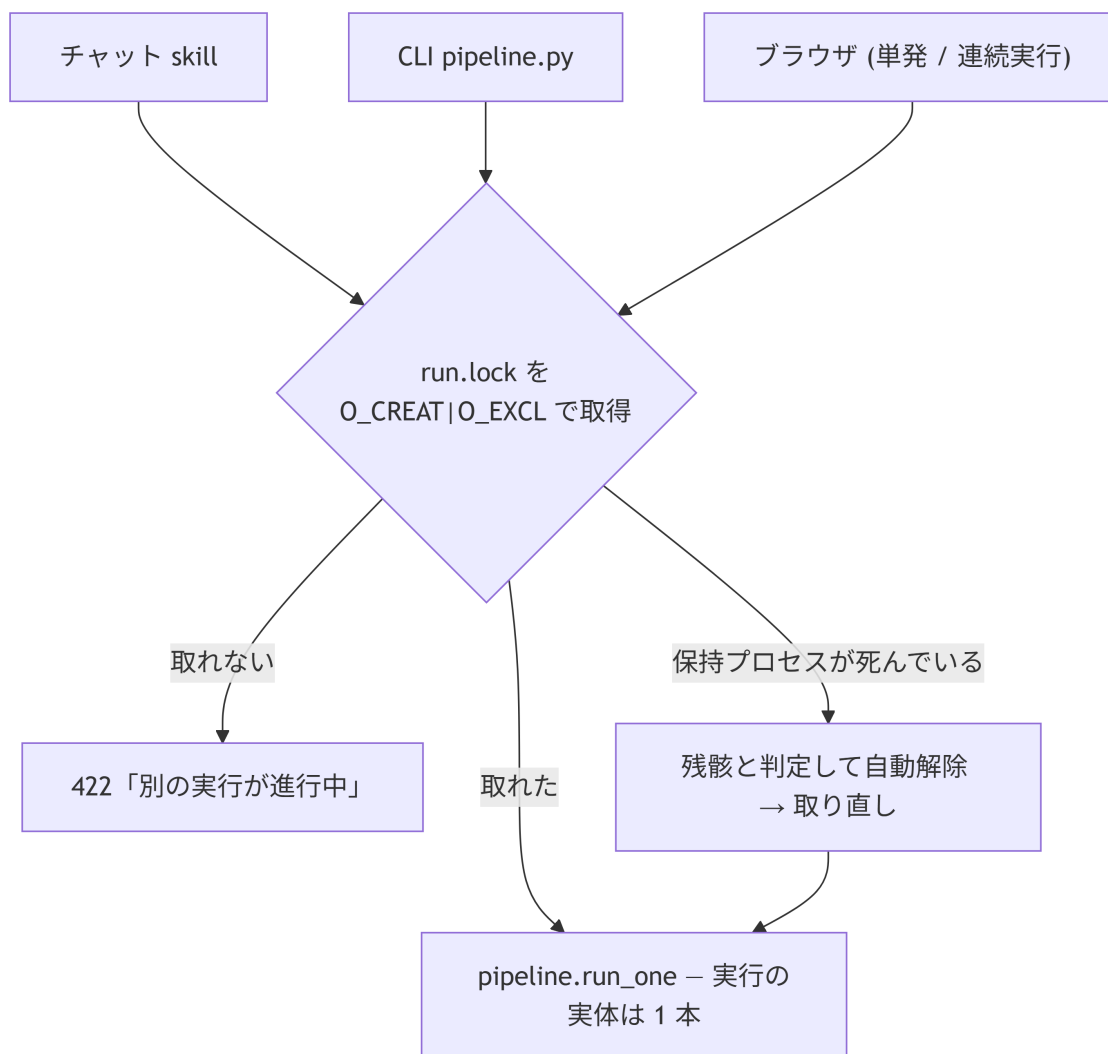


図 6.5: 3つの入口が1本の実行枠を取り合う排他

ロックは `O_CREAT|O_EXCL` の原子的なファイル作成で取る。 状態ファイルの `state` を見るだけでは「チェックしてから状態を書くまでの隙間」で二重起動できるため、チェックと予約を1操作にする。ロックにはPIDを書き、取得失敗時に保持プロセスの死活を確認して**クラッシュの残骸は自動解除する**(人が手で消す運用にしない)。

POST は**スレッドを起動して即座に応答を返す**。実行完了まで HTTP 接続を保持しない (数分かかるため)。進捗は状態ファイル経由でポーリングして受け取る。

6.7 中止・スキップの2系統

連続実行には「止める」操作が2つあり、意味が違う。

表 6.15: 停止とスキップ、止まる範囲の違い

操作	エンドポイント	効果	使う場面
停止	<code>/run-cancel</code>	バッチ全体を止める	もう今日は終わり
この件をスキップ	<code>/run-skip</code>	いまの1件だけ中止し、次へ進む	1件が長引いている

実装は `_AnyEvent`(複数 Event の OR ビュー)で、`run_one` には「バッチ全体の中止」と「この1件のスキップ」を OR にしたものを渡す。どちらが立っても実行中の headless プロセスは止まるが、**バッチのループを抜けるかどうか**が呼び出し側で分かれる。

スキップされた(関数, 工程)は失敗と同じ扱いで、そのバッチ内では再試行されない。ただし状態としては未着手のままなので、★優先で拾い直せるし、次回バッチでは自動的に対象へ戻る。

🔒 安全停止

連続3件失敗するとバッチ全体が停止する。API キー切れ・レート枯渇などの環境異常で残り全件を「失敗」として消化してしまうのを防ぐため。人がスキップした分はこの連続失敗カウントに含めない(人の操作は環境異常ではない)。

6.8 レンダリング経路

成果物(.md)がブラウザに出るまでの流れ。

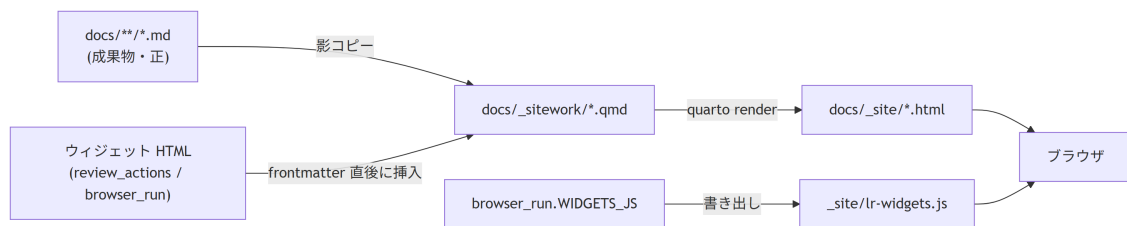


図 6.6: 成果物がブラウザに出るまでのレンダリング経路

なぜ影コピーを作るか。 Mermaid は `.qmd` でしか描画されない(`.md` のままだと図がソースのまま出る)。成果物には GitHub 流の ```mermaid` で書かせ、影コピー生成時に ```mermaid` へ変換する。`quarto render docs` を直接叩くと図が出ないのはこのため。

差分レンダリングが既定。変わったページだけを出し直すので2回目以降は数十秒で済む。ただし検索索引は更新されないため、節目で `--full` を1回実行する運用にしている。

6.9 設計判断のまとめ

表 6.16: 画面設計で採った案と、捨てた案の対比

判断	採用した理由	捨てた案
バッチ画面だけ Quarto を通さない	秒単位の更新が要る。成果物ではない	全画面 Quarto(ライブにならない)
ウィジェットをレンダ時に焼き込む	静的サイトのまま配れる。EXE 配布と両立	全画面を SPA 化(配布・オフライン閲覧が壊れる)
承認 UI を複数画面に置く	一覧からも詳細からも承認したい	単一画面に限定(往復が増える)
承認の検証はサーバ側で再実行	クライアント表示を信用しない	ボタンの活性制御だけ(改竄・競合に弱い)
残タスクは既定9件+検索	2000 関数で破綻しない	全件表示+クライアント検索(重い)
実行順番号をサーバで振る	画面の並び=実際の実行順を保証	クライアント補正(ズレる)
人待ちを運転席に集約	実際のボトルネックだから	WBS へ戻らせる(往復)

関連: 運用設計 — 導入・日常運用・配布 / 品質ゲート仕様 — 承認前に機械が何を検証するか

7. MCP サーバ仕様

7.1 設計方針(全体)

mcp-servers/legacy-reverse-mcp/server.py(FastMCP)は、skills が Bash 経由で叩いていた スクリプト層を型付きツールとして公開するサーバ。

- 判断は持たない。実体は legacy-reverse/scripts/ の各スクリプトを import または subprocess で呼ぶだけ(単体でも従来どおり動く。**二重管理なし**)
- 戻り値は構造化(dict / list)。シェル引用の事故と許可プロンプトを減らす
- 登録は**推奨だが必須ではない**。未登録環境では skill がスクリプトを直接実行する(workflow.md の規則: 登録済み環境では MCP ツール優先)
- 全ツールが root(対象プロジェクトのルート、既定 ".")を取る

7.1.1 登録(対象プロジェクトの .mcp.json)

リスト 7.1: .mcp.json への登録例

```
{
  "mcpServers": {
    "legacy-reverse": {
      "command": "python",
      "args": ["<skills リポジトリ>/mcp-servers/legacy-reverse-mcp/server.py"]
    }
  }
}
```

7.1.2 MCP 化しないもの(意図的な除外)

表 7.1: MCP 化から意図的に除外したスクリプトと理由

対象	理由
serve_site.py / browser_run.py / review_actions.py	常駐プロセスであり、かつ 人の操作の入口 (実行・承認・裁定の API)。LLM に渡すツールではない
hooks/guard_tests.py・guard_json.py	ツール呼び出しの強制ガードは hook の役割のまま(MCP 化すると迂回可能になる)

7.2 ツールリファレンス(30)

7.2.1 台帳(読み取り) — ledger.py

表 7.2: 台帳を読み取る MCP ツール

ツール	引数	返り値	説明
pipeline_status	root, func_id?	list[dict]	全関数(または指定関数)の①～⑤の進捗・stale・blocked 状態。WBS

ツール	引数	返回值	説明
			と同じ判定ロジック(<code>status_of()</code>)の生データ
<code>next_action</code>	<code>root</code>	<code>str</code>	トポロジカル順(葉から)で次に着手すべき関数とフェーズ
<code>next_actions</code>	<code>root, limit=20, phase?, skip_draft=False</code>	<code>str</code>	着手可能な関数を推奨順に最大 <code>limit</code> 件(<code>fid<TAB>次フェーズ</code>)。バッチ計画用。 <code>phase="1"~"5"</code> で絞り込み、 <code>skip_draft=True</code> で人のレビュー待ち (① <code>draft</code> ・② <code>generated</code>)を除外 — バッチ全件モードの再開時に使う (<code>draft</code> の二重書き直し防止)
<code>progress_summary</code>	<code>root</code>	<code>dict</code>	各フェーズの完了数・ <code>blocked</code> ・ <code>stale</code> ・ <code>改変</code> ・ <code>fail</code> の要約 JSON。2000 関数規模でも全関数リストを読まずに状況把握できる
<code>verify</code>	<code>root, func_id</code>	<code>dict</code>	ハッシュ連鎖(①→②、③改変、 <code>blocked</code>)の検証。 <code>ok=false</code> なら <code>problems</code> に理由
<code>next_issue_id</code>	<code>root</code>	<code>str</code>	次に使う ISSUE 番号(全体通し)

7.2.2 ⑦ 機械抽出 — `extract_fortran.py` / `extract_c.py`

表 7.3: レガシーソースを静的解析して台帳を作る MCP ツール

ツール	引数	返回值	説明
<code>extract_functions</code>	<code>root, legacy_dir="legacy", package?, write=True, infer_calls=True</code>	<code>dict</code>	Fortran ソースを静的解析し <code>functions.json</code> を生成/マージ。 再実行は常にマージ(<code>func_id</code> 不変・手修正保持) 。返回值に完全性突合の差分・推定呼び出し・未解決名の件数。詳細は <code>data/extract-report.json</code>
<code>extract_c_functions</code>	<code>root, legacy_dir="legacy", package?, write=True, infer_calls=True</code>	<code>dict</code>	C/C++ を同じ <code>functions.json</code> にマージ。 Fortran↔C の呼び出しをアンダースコア規約込みで突合 するので、抽出の実行順は問わない(後から走った側が未解決名を解決する)。監査ログは <code>data/extract-report-c.json</code>

7.2.3 機械レビュー — `review_checks.py`

表 7.4: 成果物を機械レビューする MCP ツール

ツール	引数	返り値	説明
<code>review_spec</code>	root, func_id	dict	①仕様書の機械レビュー。① draft 後・人レビュー依頼前に必ず実行し ok=true にしてから人に出す
<code>review_testspec</code>	root, func_id	dict	②テスト仕様書の機械レビュー。approved 依頼前に必ず実行
<code>review_all</code>	root	dict	全関数の①②一括レビュー(skeleton 除外)。⑥前の総点検
<code>spec_review_report</code>	root	dict	① draft の一斉レビュー表(docs/spec-review.md)を生成。生成後に render_site で HTML 化して人に案内する

検査内容の詳細は 品質ゲート仕様。

7.2.4 台帳(書き込み) — ledger.py

表 7.5: 台帳を更新し成果物を生成する MCP ツール

ツール	引数	返り値	説明
<code>add_function</code>	root, legacy_file, legacy_name, new_module?, new_name?, lines?, note?	dict	⑩の抽出漏れ・関数分割を後追いで追加。manual: true が付くので再抽出で「ソースに無い」警告が出ない(実ソースで確認されたら自動で外れる)
<code>exclude_function</code>	root, func_id, reason	dict	移植対象外にする(デッドコード等)。①～⑥・WBS・next の対象から外れ、WBS の「対象外の関数」に理由つきで残る。物理削除はしない(再抽出で別 ID として復活し成果物との紐付けが切れるため)
<code>include_function</code>	root, func_id	dict	除外を取り消して対象に戻す
<code>generate_wbs</code>	root	dict	functions.json+成果物フロントマターから docs/index.qmd(WBS)を再生成
<code>generate_skeletons</code>	root, force=False	dict	functions.json から仕様書骨子を生成(既存はスキップ)
<code>freeze_tests</code>	root, func_id	dict	③完了時にテストコードのハッシュを台帳へ記録(以降の改変を検知可能に)
<code>block</code>	root, func_id, issue_id	dict	関数を裁定待ち(🔴)にする
<code>unblock</code>	root, func_id	dict	人の裁定反映後にブロック解除し attempt をリセット

ツール	引数	返り値	説明
<code>phase_start</code>	root, phase, func_id	dict	フェーズ開始を記録(4/5の間はhookがtests/編集を拒否)
<code>phase_end</code>	root	dict	フェーズ終了を記録(hook解除)
<code>completion_check</code>	root	dict	⑥完了検証を実行し docs/completion-check.md を生成。ok=true が⑥ pass
<code>sphinx_index</code>	root	dict	functions.json から docs-sphinx/index.rst(関数一覧+トレース対応表)を再生成

7.2.5 実行系

表 7.6: テスト実行と検証を行う MCP ツール

ツール	引数	返り値	説明
<code>run_tests</code>	root, func_id	dict	⑤の一括実行: 事前 verify → pytest(tc マーカー収集)→ 結果報告書の自動生成。result: pass / fail / blocked(要裁定) / mismatch(③マーカー不整合) / verify_ng。報告書パス・実装率(rate)・attempt・blocked_by も返す
<code>check_stubs</code>	root, module	dict	④のスタブ検出。ok=true でスタブなし
<code>profile</code>	root, script?	dict	⑦の性能計測(cProfile → docs/perf.md)。script 未指定はテスト負荷のスモーク計測。ホットスポット判断は代表ワークロードで

7.2.6 出力系

表 7.7: サイト・PDF・EXE を生成する MCP ツール

ツール	引数	返り値	説明
<code>render_site</code>	root, with_sphinx=True, full=False	dict	docs/ を Quarto で HTML 化 → 続けて Sphinx API を docs/_site/api に生成 (正順を内包)。quarto 直叩きせず render_site.py を通す(Mermaid 影コピーのため)。既定は差分レンダ(変わったページだけ。2000 関数でも数十秒)。full=True のときだけ全ページ再レンダし、サイト内検索の索引も更新する
<code>build_pdf</code>	root, kind, title, output	dict	種別ごとの合本 PDF。kind: specs / test-specs / test-results

ツール	引数	返り値	説明
<code>build_viewer</code>	<code>root, name?, port?, render=True, onedir=False</code>	dict	WBS/仕様書サイト同梱の単体実行 ファイル(EXE)を <code>dist/</code> に生成。 1~2 分かかるため呼び出し側のタイムアウトに注意。 PyInstaller 必要

7.3 実装上の契約

- `run_tests` は verify NG なら pytest を実行せず `verify_ng` で返す(stale なテストの pass を防ぐ)
- `subprocess` 系の 返り値 は `{ok, exit_code, output}`(output は 末尾 4000 文字。
`PYTHONIOENCODING=utf-8` を強制し Windows のコードページ問題を回避)
- `progress_summary` は `ledger status --summary` の JSON をパースして返す (パース不能時は `{ok: false, output}` に落ちる)
- 大規模(2000 関数級)での再開は `progress_summary + next_actions` から。**全関数リストをコンテキストに読み込まない**(`pipeline_status` の全件取得は分析用途に限る)

8. 運用設計

8.1 運用モデル(全体)

人(オペレータ)の仕事は4つに限定される — トリガを引く・質問に答える・承認する・裁定する。コードや文書を直接書く必要は基本的にない(書いてよい場所は後述)。この4つはチャットからでもブラウザからでも行える。

運転モードは5経路。規模だけでなく「どこまで自動で進むか」が違う。

表 8.1: 運転モード 5 経路と、どこまで自動で進むか

モード	規模の目安	進む範囲	起動	レビューの単位
対話	1 件ずつ	1 関数 × 1 工程	<code>/Legacy-N-xxx F-xxxx</code>	1 件ずつチャットで
会話バッチ	～数十件	①を N 件	「仕様書をまとめて 10 件進めて」	一斉レビュー表でまとめて
ブラウザ単発	1 件ずつ	1 関数 × 1 工程	仕様書ページのボタン	その場のウィジェットで
ブラウザ連続実行	数十～数百件	①～⑤を工程横断で自動	バッチ実行状況ページ	同じ画面の「人待ち」から
無人バッチ (CLI)	数百～2000 件	①のみ・全件	<code>pipeline.py spec</code> (夜間実行)	翌朝、一斉レビュー表で

- どのモードでも承認が人であることは変わらない。粒度と自動化範囲の違いだけ
- ブラウザ連続実行と CLI 無人バッチは同じ実行枠を共有するので、二重に走らせても片方が待たされるだけで壊れない
- CLI の `pipeline.py` が①専用なのに対し、ブラウザ連続実行は②以降も進む。「①を全件先に流したい」なら CLI、「関数単位で完成まで持っていきたい」なら連続実行

8.2 導入手順

対象プロジェクト側のセットアップ(初回のみ):

1. skill の配置(cwd は skills リポジトリの中)

リスト 8.1: skill を対象プロジェクトへ配置する

```
cd <skills>
cp -r legacy-reverse <project>/..claude/skills/
cp -r legacy-reverse/skills/* <project>/..claude/skills/
```

2 行目は skill 発見のため(Claude Code は `..claude/skills/` 直下を見る)。以降 `<LR> = <project>/..claude/skills/legacy-reverse`、コマンドは断りがなければ `<project>` で実行する

2. **hook の登録(必須)** — `legacy-reverse/hooks/settings-example.json` を `<project>/claude/settings.json` にマージ。2 本入っている: `guard_tests.py`(PreToolUse。④⑤中の tests/ 編集を拒否)と `guard_json.py`(PostToolUse。壊れた JSON をエージェントに差し戻す)。運用中のプロジェクトに後から入れる場合は `PostToolUse` ブロックの追記だけでよい
3. **MCP サーバの登録(推奨)** — `<project>/mcp.json` に `legacy-reverse-mcp` を追加 (MCP サーバ仕様)。未登録でも動作は同じ。参照先は **skills リポジトリ側の絶対パス**なので、skill をコピーした後も リポジトリを残しておく必要がある
4. **ツール類 — Quarto(必須)**。HTML サイト生成に使う。`render_site.py` が要求するのは `quarto` バイナリだけで、`quarto-typst-pdfskill` は不要)、`pip install pytest sphinx sphinx-rtd-theme`(⑦では追加で `radon ruff bandit pip-audit`)。合本 PDF を出す場合のみ追加で: `quarto-typst-pdf/` を `<LR>` の隣にコピー (`pdf_book.py` が兄弟ディレクトリとして `qtpdf.py` を探すため) + Chromium 系ブラウザ
5. **開始** — `/legacy-reverse` でセットアップ状態を確認 → 新規なら `/legacy-0-analyze`

8.3 日常運用

8.3.1 進捗の確認と操作: WBS サイト

進捗の確認だけでなく、**実行・承認・裁定もこのサイトで完結する** (各フェーズの最後に自動更新される)。

リスト 8.2: WBS サイトの起動

```
python <LR>/scripts/serve_site.py --root .
```

- **127.0.0.1 のみに bind**(仕様書は社内資料。LAN 公開は `--host 0.0.0.0` を明示したときだけ)。LAN 公開しても書き込み系の操作はローカルからのみ受け付ける
- ポートはプロジェクト名から決まる**固定値(8100-8899)**。URL をブックマークできる。複数プロジェクト同時でも衝突しない(埋まっていれば空きへずらす)
- キャッシュ無効で配信。再レンダリング後はブラウザ更新だけで最新
- `--render` で配信前に作り直し、`--watch` で docs/ 変更を検知して自動再レンダリング。ただし **バッチ実行状況ページは Quarto を通さない**ので `--watch` に関係なくリアルタイム
- ナビバー: WBS / ドメイン知識 / 規約 / ⑥完了検証 / ⑦分析 / 新コード詳細(API) / **バッチ状況**。一斉レビュー表は navbar になく、WBS の「要対応」からリンクする

画面ごとの構成・ボタンの出現条件・HTTP API の詳細は画面設計。

8.3.2 レンダリング規則(重要)

- 各フェーズの最後に必ず `ledger wbs → render_site.py` を実行する契約 (人がブラウザで途中経過を常に確認できる状態を保つ)
- `quarto render docs` を直接叩かない。Quarto は Mermaid を .qmd でしか描けないため、`render_site.py` が影コピー(`docs/_sitework/`)を作ってから render する
- 成果物(.md)の図は GitHub 流の ```mermaid.` ```mermaid` と書くと サイト全体の render が落ちる
- Sphinx(④ API)は必ず **`render_site.py` の後**(`render_site` が `_site` を作り直すため)。MCP の `render_site` ツールはこの正順を内包している

8.3.3 承認ゲートの運用

対象: ①の reviewed 化、②の approved 化、⑤トリアージの (b)(c)、ループ上限後の再開。承認の経路は 2 系統あるが、記録先も検証も同じ。

表 8.2: 承認の 2 経路と、それぞれの流れ

経路	流れ
チャット	skill が承認用サマリ(変更点・Confidence 内訳・未回答質問)を提示 → 人が OK / 修正指示 / 保留 → skill がフロントマターを更新
ブラウザ	仕様書ページの承認ウィジェット、または一斉レビュー表の行内ボタン → サーバが機械レビューを再実行してから フロントマターを更新 → WBS・一斉レビュー表・サイトを再生成

- 勝手に approved にしない。**承認待ちで turn を終えるのは正しい動作**
- どちらの経路でも記録先は同じフロントマター(reviewed-by / reviewed-date、②は approved-*)
- 答えられない質問は「保留」でよい。ISSUE として WBS 最上部に出続ける
- draft は書き直し自由。reviewed の書き直しは②の再承認が必要になる旨を先に確認される
- ブラウザからの「修正依頼」は docs/review-feedback.md に pending として溜まり、次に AI が ①②を実行したときに自動で拾われて applied になる

8.3.4 ISSUE 運用

- 採番は全体通し(ledger next-issue)。形式は「仮説+Yes/No で答えられる問い」を強制(白紙の質問は来ない。人の判断コストを下げる)
- kind: spec-gap(④が詰まった) / domain / legacy-bug / triage(⑤ループ上限) / other
- 回答は 2 系統どちらでも同等: **チャットで回答 or ISSUE ファイルの「回答(人が記入)」欄に直接記入**
- 回答が付いたら answered → 成果物へ反映して applied。ドメイン知識は domain-knowledge.md に転記され、**同じ質問は二度と来ない**
- 全フェーズ skill は起動時に「回答済み ISSUE・規約変更」をスキャンしてから本処理に入る(ファイルに直接書いた内容は次のフェーズ起動時に自動で拾われる)

8.3.5 ⑤ fail の裁定と復帰

表 8.3: ⑤ fail の 3 分類と、人がすべき操作

分類	意味	人の操作
(a) 実装バグ	実装が①を満たしていない	何もしない(AI が④修正 → ⑤再実行を自走)
(b) テストコードバグ	③が②とズレている	ISSUE の証拠を見て承認 → ③再生成
(c) 仕様が誤り	②①自体が間違い	ISSUE で裁定 → ①改訂を指示(伝搬は自動検知)

(a) のループは **3 回で自動停止**し、triage ISSUE 起票+WBS ➊。復帰手順は 2 通り。

チャット経路(3 ステップ):

- ISSUE を読んで (a)/(b)/(c) を裁定し回答(チャットまたは回答欄)
- AI が反映(applied)したら unblock(「F-xxxx をアンブロックして」でよい)
- /legacy-5-test F-xxxx を再トリガ(attempt は 1 からやり直し)

ブラウザ経路(1 操作 + 1 クリック): 裁定待ちの関数の仕様書ページ、または バッチ実行状況ページの「人待ち」タブに ISSUE の質問がそのまま出る。回答を書いて送ると「ISSUE への記入(answered)→ unblock → サイト更新」まで完了し、リロード後に出る「⑤を実行する」ボタンで再テストできる。

8.4 無人バッチ実行

全件実行(数百～2000 件)は**エージェントの会話内ループでは行わない**(コンテキスト上限・コンパクション劣化)。無人ドライバを使う。入口は2つある。

表 8.4: 無人実行の2入口と、向く場面の違い

入口	カバー範囲	件数指定	コスト上限	向く場面
CLI <code>pipeline.py spec</code>	①のみ	<code>--max-funcs</code>	<code>--budget-usd</code>	夜間に①を全件流す
ブラウザ「連続実行」	①～⑤	プルダウン (全部 / 1 / 10 / 50 / 100)	UIからは指定しない	承認しながら関数を完成まで進める

どちらも実行の実体(`run_one`)と実行枠ロックを共有するので、同時には走らない。ブラウザ側でコスト上限を外したのは、件数指定で足りる上に、コストは画面の指標カードで常時見えているため(画面設計)。

8.4.1 CLI ドライバ(`pipeline.py`)

リスト 8.3: CLI ドライバの主な起動パターン

```
python <LR>/scripts/pipeline.py spec --root . # 全件 draft まで無人実行
python <LR>/scripts/pipeline.py spec --root . --max-funcs 200 # 今日は 200 件だけ
python <LR>/scripts/pipeline.py spec --root . --budget-usd 20 # コスト上限
python <LR>/scripts/pipeline.py spec --root . --dry-run # 対象の確認のみ
```

8.4.2 実行手順(ランブック)

エージェント(headless Claude)を機械的に回すための通し手順。

1. 前提を確認する

- `claude --version` が通ること(PATH に無ければ `--claude-cmd <パス>` で明示)
- 対象プロジェクトに skill 一式が配置済みであること(導入手順 1～2)
- **headless は許可プロンプトに答えられない**。①が使うツール(Read/Write/Edit と スクリプト実行)を `.claude/settings.json` の `permissions.allow` で許可しておくか、信頼できる閉じた環境に限り `--skip-permissions` (= `--dangerously-skip-permissions`)を明示する

2. 対象と実行内容を確認する(`dry-run`)

リスト 8.4: 実行前に対象とコマンドを確認する

```
python <LR>/scripts/pipeline.py spec --root . --dry-run
```

対象件数と、1 件ごとに起動される `claude` コマンドがそのまま表示される (既定: `claude -p "/legacy-1-spec F-xxxx" --output-format json --max-turns 50`)

3. 起動する

リスト 8.5: 無人バッチの起動

```
python <LR>/scripts/pipeline.py spec --root . [--max-funcs N] [--budget-usd X] [--model M]
```

4. ドライバが1関数ごとに行うこと(人の介入なし):

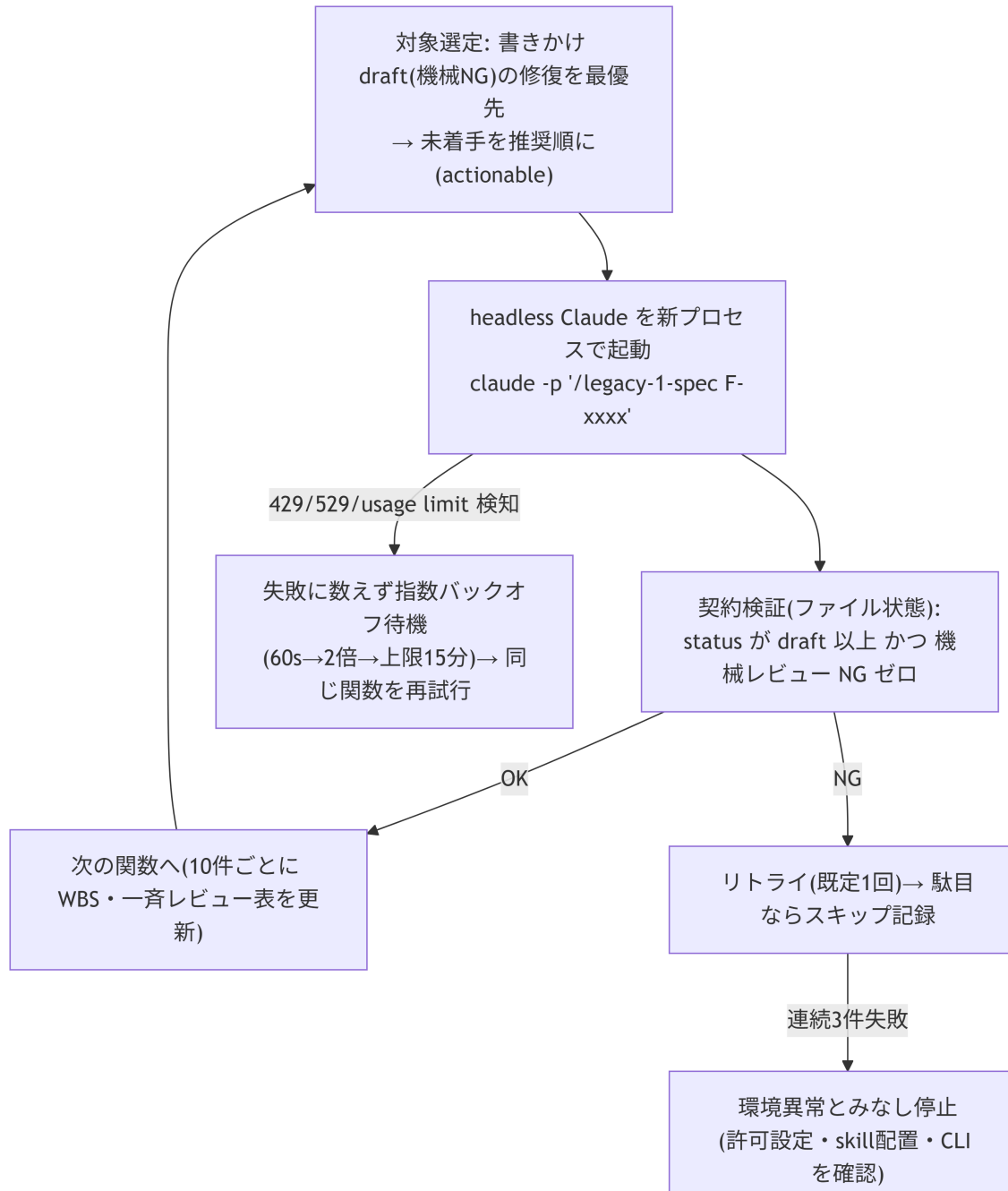


図 8.1: ドライバが1関数ごとに回す判定と待機

5. 監視する(任意。放置してよい)

- 端末出力: [n] F-xxxx draft化 OK(累計 \$X.XX) / 検証 NG・待機の理由
- `.legacy-reverse/pipeline-log.jsonl`: 1行1関数(結果・attempt・所要秒・コスト)
- ブラウザ: `serve_site.py --watch`を立てておけばWBSと一斉レビュー表がチャックごとに自動更新される

6. 止まる条件(いずれも安全停止。再開は同じコマンド)

- `--max-funcs` / `--budget-usd` の上限到達
 - 連続失敗が `--max-consecutive-fail`(既定 3)に到達 = 環境異常の疑い
 - レート待機の累計が `--rate-wait-total`(既定 6 時間)を超過
7. **中断・再開** — Ctrl-C・電源断はどこでも安全。同じコマンドで続きから再開 (処理済み draft は再選定されず、書きかけ draft は機械 NG として先に修復される)
 8. **翌朝の人の作業** — docs/spec-review.md(一斉レビュー表)を見て「全部 OK / F-xxxx は修正: ~」を返す → reviewed 化。失敗として スキップされた関数はログを確認して個別に / legacy-1-spec F-xxxx に対応

8.4.3 オプション一覧

表 8.5: CLI ドライバのオプションと既定値

オプション	既定	意味
<code>--max-funcs N</code>	0(無制限)	このセッションで処理する上限件数
<code>--budget-usd X</code>	0(無制限)	累計コスト上限。超えたら停止
<code>--chunk N</code>	10	WBS・一斉レビュー表を更新する間隔
<code>--retries N</code>	1	契約検証 NG 時のリトライ回数
<code>--max-consecutive-fail N</code>	3	連続失敗でドライバを停止する閾値
<code>--max-turns N</code>	50	1 関数あたりのエージェントターン上限
<code>--timeout N</code>	1800	1 関数あたりの秒数上限(ハング対策。超過は通常リトライ経路)
<code>--backoff-base</code> / <code>--backoff-max</code>	60 / 900	レートリミット検知時の待機秒(初期値→2 倍ずつ→上限)
<code>--rate-wait-total N</code>	21600	レート待機の累計上限秒。超えたら安全停止
<code>--pause N</code>	0	関数間の予防的待機秒(レートリミットに当たりやすい環境用)
<code>--model M</code>	—	claude に渡すモデル指定
<code>--skip-permissions</code>	off	<code>--dangerously-skip-permissions</code> を渡す(信頼できる環境のみ)
<code>--claude-cmd</code> / <code>--claude-args</code>	PATH 検索 / —	claude 実行ファイルの明示 / 追加引数
<code>--prompt-template</code>	/legacy-1-spec {fid}	1 関数あたりのプロンプト({fid} が置換される)
<code>--dry-run</code>	off	実行せず対象とコマンドを表示
<code>--no-render</code>	off	終了時の HTML 再生成を省略

8.4.4 設計上の性質

- **1 関数 = 1 つの新しい headless Claude プロセス**(`claude -p "/legacy-1-spec F-xxxx"`)。コンテキストが積み上がり、1 関数あたりのトークンは常に一定 — トークン上限に依存しない
- 完了は LLM の申告でなく **ファイル状態で契約検証**(status: draft + 機械レビュー NG ゼロ)。NG はリトライ → スキップ記録、連続失敗で安全停止
- **タイムアウト・レートリミット耐性**: 1 関数ごとに秒数上限で打ち切り(ハング対策)。レートリミット / 利用枠上限は失敗に数えず指数バックオフで待機 → 同じ関数から自動再開 (利用枠の時間リセットを人手なしで跨げる。待機累計の上限超過で安全停止)

- ・チャンクごとに WBS・一斉レビュー表を自動更新。人は溜まった draft を随時レビューしてよい(全件完了を待つ必要はない)
- ・中断(Ctrl-C・電源断)はどこでも安全。同じコマンドで続きから再開(処理済み draft は二重に書き直さない — 再開時は `next_actions(skip_draft=True)` 相当の選定)
- ・実行ログ: `.legacy-reverse/pipeline-log.jsonl`(関数別の結果・コスト・所要)
- ・前提: 対象プロジェクトに skill 配置済み、headless 用に必要ツールを `.claude/settings.json` で allow(または `--skip-permissions` を明示)

典型的な運用: 夜間に回して、翌朝一斉レビュー表(spec-review.md)を見る。

8.5 中断・再開

進捗の正はすべてファイル側(functions.json / ledger.json / フロントマター)にあり、会話コンテキストには無い。どのタイミングで中断しても、再開手順は常に同じ 3 コマンド:

リスト 8.6: 中断後の再開に使う 3 コマンド

```
ledger status --summary          # 全体状況(2000 関数でも数行)
ledger next --all --limit 20     # 着手可能な関数と次フェーズ
python <LR>/scripts/review_checks.py all --root .  # 成果物の健全性
```

チャットなら `/legacy-reverse` と打つだけ(上記を実行して次の一手を提案してくる)。

- ・やり直し・再列挙は禁止。全関数を列挙した長大な出力をコンテキストに読み込まない
- ・①の再実行も抽出器がマージ動作なので安全
- ・「どこまで進んだか分からない」ときは WBS を開くのが最短(🚫🔴🔴🔴 が「要対応」として最上部)

8.6 大規模運用(200 関数超)

- ・WBS のトップは自動でダッシュボードに切り替わる(進捗サマリ・要対応・人への質問・次の一手・レガシーファイル別進捗)。全関数明細は docs/wbs/ のファイル別サブページへ分割
- ・2000 関数でも「いま何が問題で、次に何をやるか」はトップ 1 画面で分かる
- ・状況把握は `progress_summary / next_actions`(MCP)または `summary / next`(CLI)で。全関数表を読む運用はしない

8.7 配布・出力

8.7.1 レビューアへの配布(単体実行 EXE)

サーバ用意や社内公開なしで、サイト同梱の実行ファイルを渡し各自のローカルホストで開く。

リスト 8.7: 単体実行 EXE のビルド

```
pip install pyinstaller          # 初回のみ
python <LR>/scripts/build_viewer.py --root .  # → dist/<プロジェクト>-wbs.exe
```

- ・受け手はダブルクリックだけ。Python も Quarto も不要(サイト込み 10MB 程度)
- ・外部フォント参照はビルド時に除去 — 閉域ネットワークでも開け、外部通信も飛ばない
- ・中身はビルド時点のスナップショット。進捗が動いたら作り直して渡し直す(`--no-render` で再レンダリング省略可)
- ・クロスコンパイル不可: Windows 用 .exe は Windows 上でビルド
- ・SmartScreen 等に止められる環境は `--onedir`(フォルダ配布)、サイト別配布なら `--no-embed`

8.7.2 PDF 出力(種別ごとの合本)

リスト 8.8: 種別ごとの合本 PDF 出力

```
python <LR>/scripts/pdf_book.py specs --root . --output pdf/関数仕様書.pdf --
title 関数仕様書
python <LR>/scripts/pdf_book.py test-specs --root . --output pdf/テスト仕様書.pdf --
title テスト仕様書
python <LR>/scripts/pdf_book.py test-results --root . --output pdf/テスト結果報告書.pdf
--title テスト結果報告書
```

- quarto-typst-pdf skill に委譲(Quarto 1.10 系 book+typst の既知不具合 2 件を前処理とパッチで回避)
- PDF では相対リンク・アンカーは平文化される(リンクは HTML 版に残る)
- 生成後は `qtpdf.py check <pdf>` で豆腐・はみ出しを機械チェック
- Mermaid の PDF 化には Chromium 系ブラウザが要る(`qtpdf.py doctor` で確認)

8.8 手編集の可否

表 8.6: ファイル別の手編集可否と、その理由

ファイル	手編集	説明
docs/domain-knowledge.md ・ docs/conventions.md	○	人が著者。業務知識・規約はここへ
ISSUE の「回答(人が記入)」欄	○	じっくり書きたい裁定はファイルで
docs/specs/(仕様書)	△	編集可。ハッシュ連鎖が下流を stale にして再確認が走る(設計どおり)
docs/review-feedback.md	△	ブラウザが追記し skill が applied 化する。人が直接足してもよいが書式を守る
WBS(index.qmd) ・ test-results/ ・ completion-check.md ・ spec-review.md ・ quant.md ・ perf.md	✗	自動生成。次の再生成で消える
data/functions.json	△	正データ。編集するなら JSON を壊さないこと (壊れると全スクリプトが停止する。hook が AI の編集は検査する)
data/ledger.json	✗	スクリプト専用
.legacy-reverse/ 配下すべて	✗	実行時データ。pipeline-status.json は表示専用、batch-priority.json は画面が書く

conventions.md を途中で変更した場合、skill が変更を検知して「どの関数の成果物と不整合になり得るか」を洗い出して人に報告する契約。

8.9 トラブルシューティング(運用者向け要約)

表 8.7: よくある症状と、その意味と対処

症状	意味	対処
WBS に  ISSUE-xxx	④⑤ループ上限、裁定待ち	ISSUE に回答 → unblock → ⑤再トリガ
WBS に stale△	①改訂で②が古い	/legacy-2-testspec で再生成 → 再承認
WBS に △改変	freeze 後にテストが変わった	意図した変更なら③で再 freeze。心当たりがなければ調査
tests/ 編集が拒否された	hook が正常動作	テスト側の疑義は ISSUE 経由(正規経路)
再レンダリング失敗	配信サーバが _site をロック	serve_site.py を使う(cwd を動かさないでロックしない)
図がソースのまま表示	quarto render を直叩きした	render_site.py で出し直す
①②の機械レビュー NG 報告	ハルシネーション・省略を検知(正常動作)	AI が直して再提出してくる。NG 付き成果物を承認しない
⑥で完全性突合 NG	抽出器の 2 系統カウント不一致(変則ソース)	extract-report.json の該当ファイルを AI に調査させる。判断がつかなければ ISSUE
バッチが「データ異常のため停止」	正データ(functions.json 等)が JSON として壊れた	停止理由に行・列が出る。 <code>git diff</code> で直前の変更を確認して修復、または <code>git checkout --</code> で戻す。hook 登録済みならこの状態になる前に AI へ差し戻される
連続実行が「連続 3 件失敗」で停止	環境異常の疑いで安全停止 (1 件ずつの失敗とは別扱い)	結果パネルの NG 行から応答ログを読む。API キー・レート・claude の解決を確認して再開始
「別の実行が進行中」と断られる	実行枠ロックを双方向で取り合う設計(正常動作)	完了を待つか停止する。異常終了の残骸なら次の実行時に自動解除される
画面がずっと「実行中」のまま	実行プロセスが異常終了した残骸	PID の死活で自動的に「停止(異常終了)」へ読み替えて表示される。そのまま再実行してよい
配布 EXE で承認ボタンが効かない	EXE はスナップショットなので書き込み系を 403 で拒否する(仕様)	生成元のプロジェクトを <code>serve_site.py</code> で開いて操作する
サイト内検索に新しいページが出ない	差分レンダは検索索引を更新しない	節目で <code>render_site.py --root . --full</code> を 1 回実行する

症状	意味	対処
配布 EXE が古い	ビルド時点のスナップショット	build_viewer.py で作り直す
PDF だけ図が出ない	Mermaid 画像化に Chromium が要る	qtpdf.py doctor で確認。 HTML は影響なし

より詳しい一覧は MANUAL.md のトラブルシューティング章を参照。